FPT Education

FPT UNIVERSITY

Nốt

One positive note every day

Đoàn Thị Kim Thúy · SE190992

Phùng Thế Quân · SE193519

Phan Đức Duy · SE193591

Phan Hữu Phước · SE161951

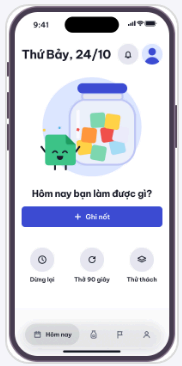


Introduction

Tiny Wins (Nốt) is a mobile app for burned-out final-year students like Minh (22): every evening they drop one small win into a jar instead of staring at an unfinished to-do list. After a rough day, the jar hands back an old win to remember. Small bravery and movement challenges, a one-tap self-control log and a guided journal help them rebuild confidence, one note at a time. Success criterion: log a mood and one win in under 30 seconds.

Overview

From the Lab 2 design to a cloud-connected Flutter app



S02 Hôm nay
↓
/today
↓
TodayScreen
↓
TodayViewModel

→

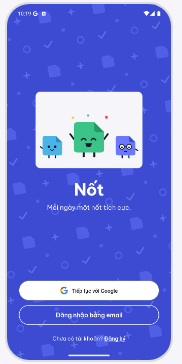
View
TodayScreen

ViewModel · Riverpod
TodayViewModel

Service · interface
NoteRepository

Firebase · lib/firebase
FirestoreNoteRepository

→



Auth
Firestore
Cloudinary

1 Lab 2 design → code

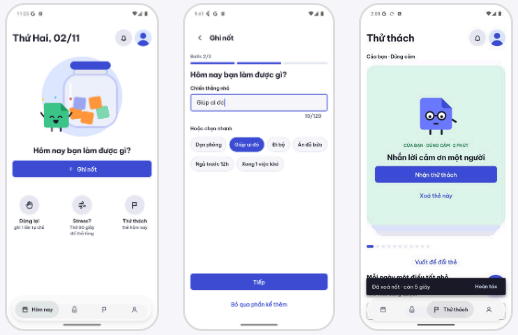
Each Figma screen mapped to a route, Screen and ViewModel

2 MVVM

Screens never touch the SDK; only lib/firebase does

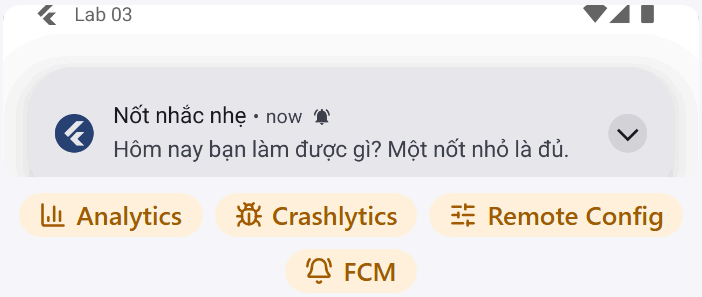
3 Firebase, round 1

Sign-in, notes in Firestore, photos on Cloudinary

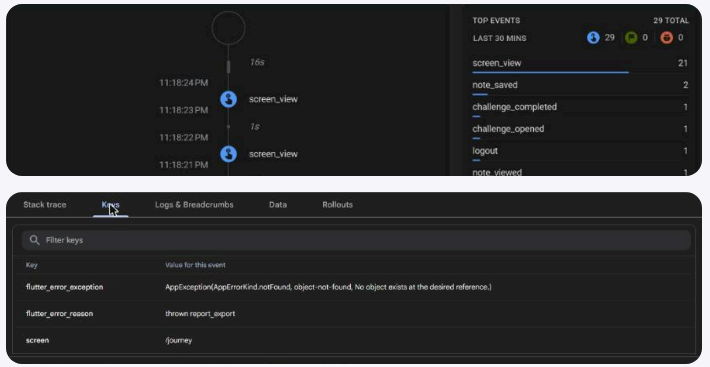


S02 S03 S07

→



→



4 Features

All 12 screens; flows F1 to F3 with their error states

5 Firebase, round 2

Events, crashes, feature flags and push messages

6 Evidence

Console, DebugView, Crashlytics, ARTEMIS · evidence/

Design system in code

Lab 2 tokens become one ThemeData; each Lab 2 component is one reusable widget

FPT Education
FPT UNIVERSITY

Tokens → ThemeData

Colour → `ColorScheme` app_colors.dart

primary	primary-container	surface	on-surface	error	success	heat/3	heat/5
#3D4FD6	#DDE1FF	#F6F7FB	#1B1D2A	#B3261E	#1F9D63	#FFD24A	#F0524B

Type → `TextTheme` app_typography.dart · Be Vietnam Pro

Nốt nhắc bạn — Display 44 · `displayMedium`

Nốt nhắc bạn — Headline 28 · `headlineMedium`

Nốt nhắc bạn — Title 18 · `titleMedium`

Nốt nhắc bạn — Body 16 · `bodyLarge`

Nốt nhắc bạn — Label 14 · `labelLarge`

Space → `AppSpace`

4 8 12 16 24

Radius → `AppRadius`

8 14 note 16 card full

`AppTheme.light()` wires all of it into one `ThemeData`; screens read the theme, never a raw colour or size

Lab 2 components → widgets/

Button · 5 variants × 4 states	→	`AppButton(variant:, loading:)`
Text field · error, disabled	→	`AppTextField(errorText:)`
Card/Nốt · 4 note types	→	`NoteCard(type:)`
BottomNav · 4 tabs	→	`AppBottomNav(currentIndex:)`
Dialog · confirm, destructive	→	`showAppDialog(kind:)`
Loading · Empty · Error	→	`LoadingState · EmptyState · ErrorState`
MoodOption · selected	→	`MoodOption(mood:, selected:)`
Mascot · 8 states (below)	→	`Mascot(state:)`
Illustration/Hũ chiến thắng	→	`JarIllustration(notes:)`



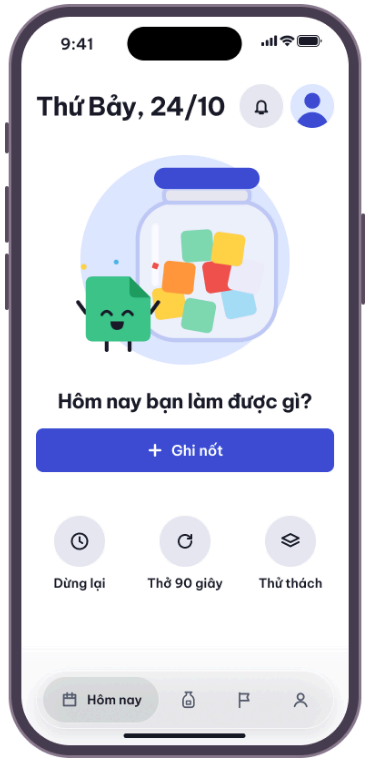
Tụi Nốt: one face per mood and app state, so mood is never colour alone

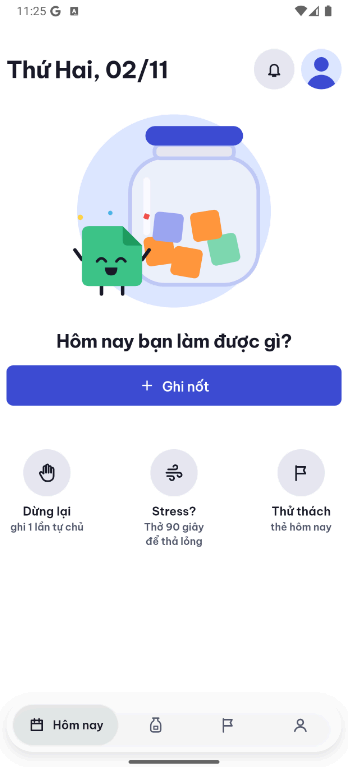
docs/design-traceability.md §3 components, §4 tokens · lib/theme/ · lib/widgets/

Figma vs app

FPT Education
FPT UNIVERSITY

Today

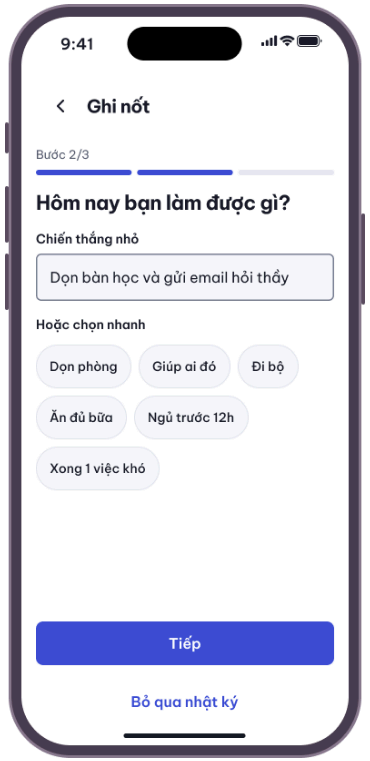




Figma | App

- Android status bar (D-04)
- New shortcut icons and captions (D-25)
- "Ghi nốt" asks win or journal first (D-29)

Write a note





Figma | App

- "Bỏ qua nhật ký" is now "Bỏ qua phần kể thêm" (D-28)
- 0/120 counter under the field (D-02)
- AI free-tell is planned, not built

Stats





Figma | App

- Compared with the previous 30 days (D-18)
- Chart scales to the card width (D-09)
- Weekly goal opens the weekly card (D-06)

docs/design-traceability.md · §1 screens, §5 every deviation with its reason

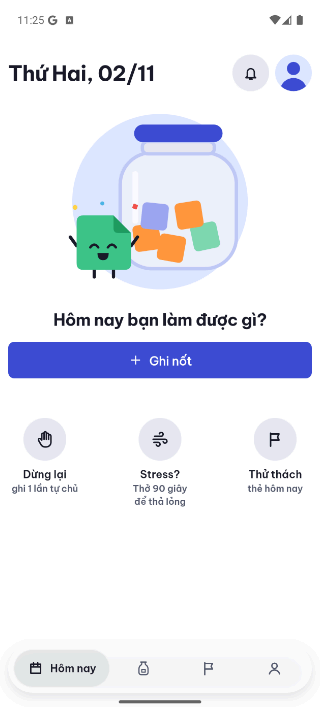
FPT Education
FPT UNIVERSITY

Riverpod + MVVM

Riverpod is a Flutter library that keeps app state and services in providers; a widget watches a provider and rebuilds when it changes.

- `Provider`: one service, typed by its interface
- `NotifierProvider`: one ViewModel and its `ViewState<T>`
- `ProviderScope(overrides:)`: Firebase in the app, fakes in tests

In Nốt: TodayScreen watches TodayViewModel, which reads NoteRepository.



S02 Today: drawn from the state of TodayViewModel

FPT Education
FPT UNIVERSITY

Riverpod + MVVM

WHY WE USE IT

Screens never touch Firebase

View → ViewModel → service interface; only `lib/firebase/` imports the SDK.

Tests without Firebase

Tests override the same providers with fakes.

Every state is explicit

`Loading | Empty | Failure | Data`, so each screen draws errors too.

WHY NOT SOMETHING ELSE?

✕ Only `setState`

Logic lives inside widgets, so it cannot be tested without the UI.

✕ Provider (package)

Reads need a `BuildContext`, and a missing provider fails only at run time.

✕ Bloc

An event class and a state class for every screen: more code for the same MVVM split.

FPT Education
FPT UNIVERSITY

Riverpod + MVVM

Riverpod needs no cloud setup: add the package, wrap the app in one ProviderScope that wires every Firebase implementation, and let tests wire fakes the same way.

```
26      flutter_local_notifications: ^22.3.3
27      flutter_riverpod: ^3.4.3
28      flutter_svg: ^2.3.0
```

1 Add `flutter_riverpod` to pubspec.yaml

```
99    runApp(
100     ProviderScope(
101       overrides: [
102         authServiceProvider.overrideWithValue(auth),
103         messagingServiceProvider.overrideWithValue(messaging),
104         analyticsServiceProvider.overrideWithValue(analytics),
105         remoteConfigServiceProvider.overrideWithValue(remoteConfig),
106         userRepositoryProvider.overrideWithValue(FirestoreUserRepository()),
107         noteRepositoryProvider.overrideWithValue(
108           FirestoreNoteRepository(currentUid: uid),
```

2 Wrap the app in `ProviderScope` and override each service

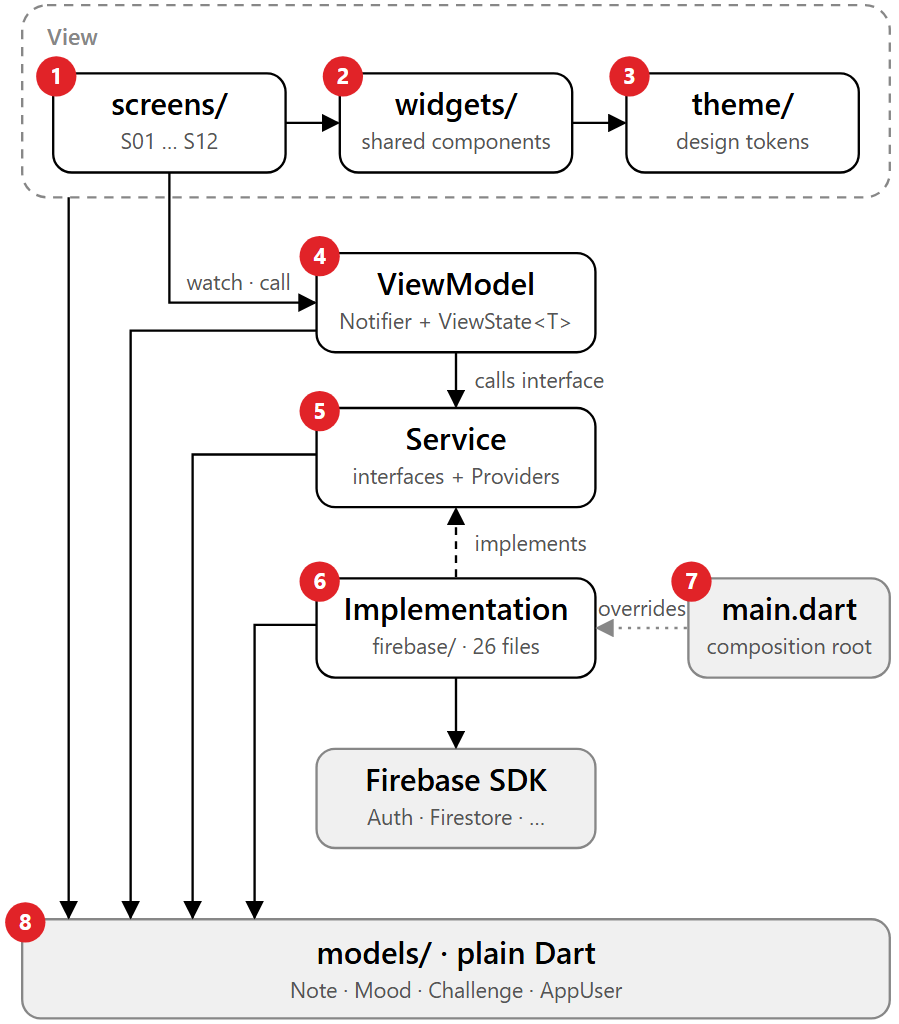
```
20    ProviderContainer makeContainer({bool smallMilestone = false}) {
21      final c = ProviderContainer(
22        overrides: [
23          authServiceProvider.overrideWithValue(
24            FakeAuthService(signedIn: testUser),
25          ),
26          noteRepositoryProvider.overrideWithValue(notes),
27          clockProvider.overrideWithValue(clock),
28          // Mốc tháng nhỏ để dễ chạm mốc trong test.
```

3 Tests override the same providers with fakes

View demo and evidence →

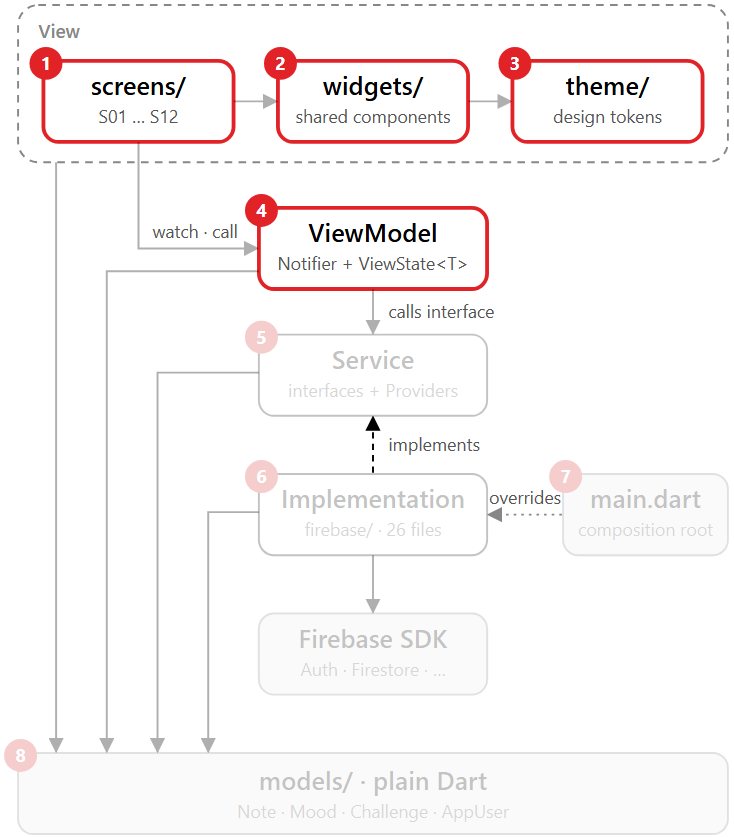
Riverpod + MVVM

```
lib/
├── main.dart                    composition root          7
├── models/note.dart             plain Dart                8
├── services/
│   └── note_repository.dart     interface+Provider        5
├── firebase/
│   └── firestore_note_repository.dart                     6
├── viewmodels/
│   ├── view_state.dart          Loading | Empty | Failure | Data
│   └── today_view_model.dart    Notifier                  4
├── screens/today/
│   └── today_screen.dart        switch on state           1
├── widgets/app_button.dart      shared component          2
└── theme/app_tokens.dart        → packages/core           3
```



Numbers ①–⑧ are the layers; the next two slides show the code of each one · docs/architecture.md

Riverpod + MVVM



1 screens/ watches its ViewModel, switches on the state

```
40        });
41        final state = ref.watch(todayViewModelProvider);
42        final hasDraft = ref.watch(hasDraftProvider).value ?? false;
43        final summary = switch (state) {
44          Data<TodaySummary>(:final value) => value,
45          _ => null,
46        };
```

lib/screens/today/today_screen.dart :41-45

2 widgets/ one shared component, variant as a parameter

```
10    /// chạm chạm. Mỗi màn chỉ nên có một nút [AppButtonVariant.filled].
11    class AppButton extends StatelessWidget {
12      const AppButton({
13        required this.label,
14        required this.onPressed,
15        this.variant = AppButtonVariant.filled,
16        this.icon,
```

lib/widgets/app_button.dart :11-15

3 theme/ spacing tokens, no hard-coded numbers

```
3     /// Figma collection **Size** (`space/*`) và quy ước lề của Lab 2.
4     abstract final class AppSpace {
5       static const double s2 = 2;
6       static const double s4 = 4;
7       static const double s6 = 6;
8       static const double s8 = 8;
9       static const double s10 = 10;
10      static const double s12 = 12;
11      static const double s14 = 14;
```

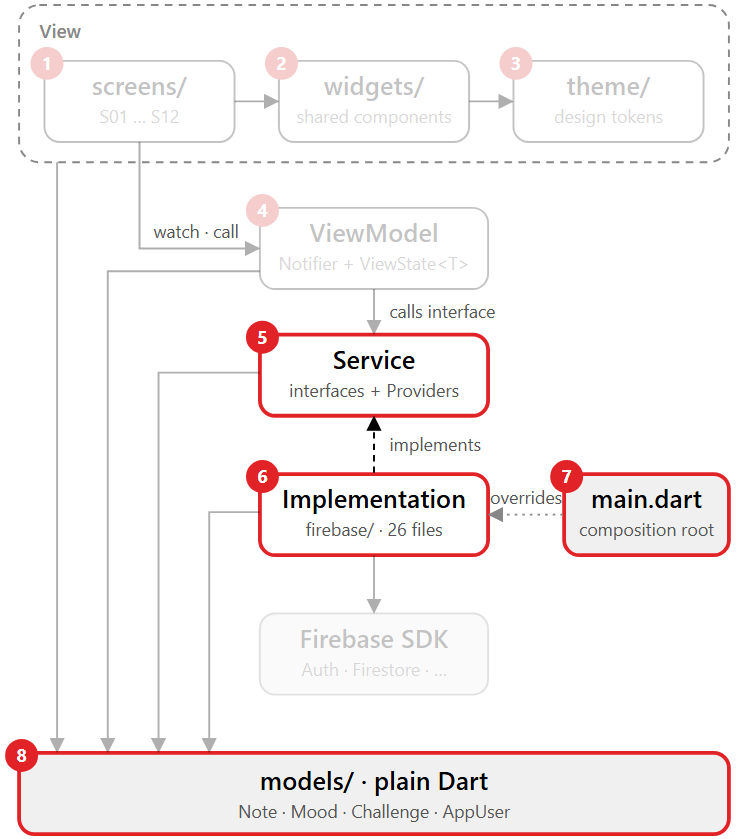
packages/core/lib/theme/app_tokens.dart :4-10

4 ViewModel `NotifierProvider` + `Notifier<ViewState>`

```
55    final todayViewModelProvider =
56        NotifierProvider<TodayViewModel, ViewState<TodaySummary>>(
57          TodayViewModel.new,
58        );
59
60    /// ViewModel của S02. Theo dõi nốt từ đầu tháng (hoặc từ hôm qua nếu hôm nay
61    /// là mùng 1) nên hũ tự cập nhật khi có nốt mới.
62    class TodayViewModel extends Notifier<ViewState<TodaySummary>> {
63      /// Chưa có dữ liệu sau chừng này thì chuyển sang Error state (T3 → T2).
```

lib/viewmodels/today_view_model.dart :55-62

Riverpod + MVVM



5 Service the interface the ViewModel calls

```dart
/// nốt nằm trên máy với `pendingSync = true` và tự đồng bộ sau.
abstract interface class NoteRepository {
  /// Các nốt tạo từ [since] trở đi, mới nhất trước. Phát lại khi có nốt mới
  /// hoặc khi một nốt đồng bộ xong.
  Stream<List<Note>> watchSince(DateTime since);

  /// [limit] nốt mới nhất (trang đầu của S05), phát lại như [watchSince].
  Stream<List<Note>> watchRecent({int limit = 30});
```

lib/services/note_repository.dart :9-15

6 Implementation implements it with Firestore

```dart
/// ngay trên máy, rồi tự gửi lên server khi có mạng.
class FirestoreNoteRepository implements NoteRepository {
  FirestoreNoteRepository({
    required this.currentUid,
    FirebaseFirestore? firestore,
  }) : _firestore = firestore ?? FirebaseFirestore.instance;
```

lib/firebase/firestore_note_repository.dart :19-23

7 main.dart overrides the provider with the Firestore class

```dart
    userRepositoryProvider.overrideWithValue(FirestoreUserRepository()),
    noteRepositoryProvider.overrideWithValue(
      FirestoreNoteRepository(currentUid: uid),
    ),
    groupRepositoryProvider.overrideWithValue(
```

lib/main.dart :107-109

8 models/ plain Dart, shared by every layer

```dart
/// Một tờ nốt trong hũ, tài liệu `users/{uid}/notes/{id}`.
class Note {
  const Note({
    required this.id,
    required this.type,
    required this.title,
    required this.createdAt,
    this.mood,
    this.body = '',
    this.pendingSync = false,
```

lib/models/note.dart :7-14

```
$ grep -rl "package:firebase" lib/screens lib/widgets lib/viewmodels
```
→ 0 files: only lib/firebase uses the SDK

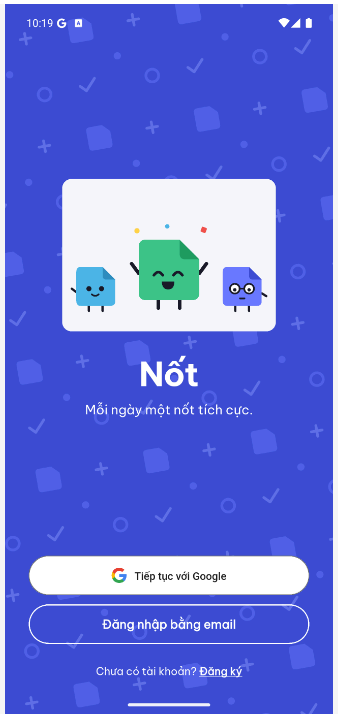
FPT Education
FPT UNIVERSITY

Firebase Authentication

Firebase Authentication is Google's hosted sign-in service: it checks who the user is and gives each account a stable user ID (UID).

- Providers: Google (`google_sign_in`) and email + password
- `authStateChanges()` streams signed in / signed out
- Failures return codes, e.g. `network-request-failed`

In Nốt: Google or email sign-in on Welcome.



Welcome offers two sign-in methods

FPT Education
FPT UNIVERSITY

Firebase Authentication

WHY WE USE IT

The UID is the key of all data

Notes live under `users/{uid}/…`; rules allow only that user.

No passwords on our side

Google keeps the credentials; the app never stores one.

Routing follows the session

Signed out → `/welcome`, signed in → `/today`.

WHY NOT SOMETHING ELSE?

✕ Our own login server

We would have to store and hash passwords and keep a server running.

✕ Supabase Auth / Auth0

A second vendor; Firestore rules only know Firebase users (`request.auth`).

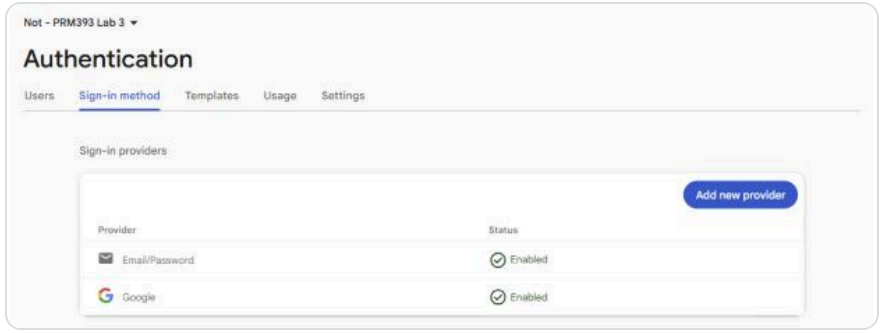
✕ Google Sign-In alone

Gives a Google account, but no UID that Firestore rules can check.

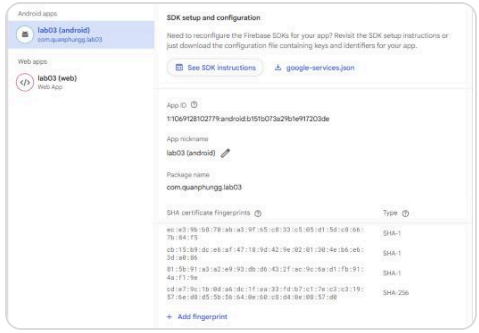
FPT Education
FPT UNIVERSITY

Firebase Authentication

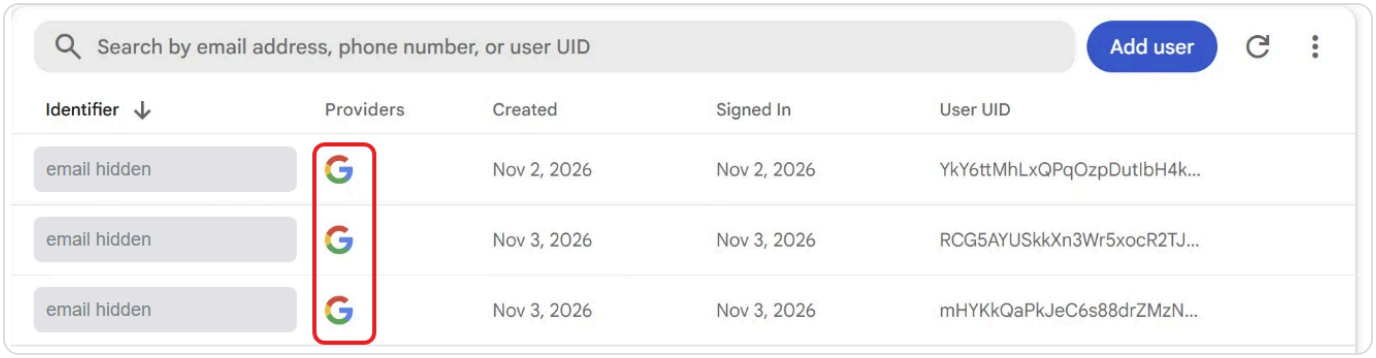
Enable the two providers, register the app's SHA fingerprints so Google sign-in works on Android, then check that real accounts appear with their UID.



1 Authentication › Sign-in method: enable Google and Email/Password



2 Project settings › Android app: add the SHA-1 / SHA-256 fingerprints



3 Authentication › Users: accounts and their UIDs

View demo and evidence →

Firebase Authentication

lib/firebase/firebase_auth_service.dart :47–58

Google ID token → Firebase credential

```dart
      ));
      final account = await _google.authenticate();
      final idToken = account.authentication.idToken;
      if (idToken == null) {
        throw const AppException(
          AppErrorKind.unknown,
          code: 'missing-id-token',
          detail: 'Google không trả idToken; kiểm tra SHA-1 và web client ID.',
        );
      }
      final result = await _auth.signInWithCredential(
        GoogleAuthProvider.credential(idToken: idToken),
      );
      return _toAppUser(result.user)!;
```

lib/firebase/firebase_auth_service.dart :119–131

Error code → message kind

```dart

  static AppException _map(FirebaseAuthException e) {
    final kind = switch (e.code) {
      'network-request-failed' || 'too-many-requests' => AppErrorKind.network,
      'invalid-credential' ||
      'invalid-email' ||
      'wrong-password' ||
      'user-not-found' ||
      'email-already-in-use' ||
      'weak-password' ||
      'user-disabled' => AppErrorKind.invalidInput,
      _ => AppErrorKind.unknown,
    };
    return AppException(kind, code: e.code, detail: e.message);
  }
```

lib/router/auth_redirect.dart :25–29

Signed out → Welcome, signed in → Today

```dart


    final signedIn = auth.hasValue && auth.value != null;
    if (!signedIn) {
      return onWelcome ? null : _withNext(AppRoutes.welcome, next);
    }
    if (onSplash || onWelcome) return next ?? AppRoutes.today;
    return null;
```

lib/viewmodels/auth_view_model.dart :116–127

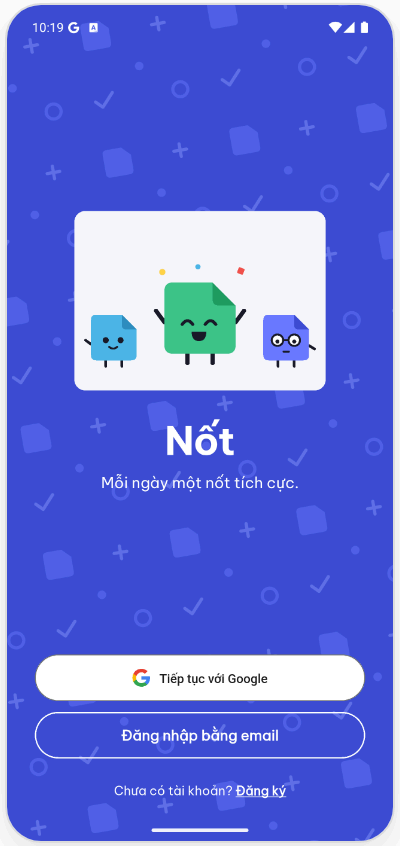
Sign-out clears local data and the FCM inbox

```dart
  /// Đăng xuất và xoá dữ liệu của người dùng còn trên máy.
  Future<void> signOut() => _run(AuthAction.signOut, () async {
    final analytics = ref.read(analyticsServiceProvider);
    await analytics.track(AnalyticsEvents.logout);
    await analytics.setUserId(null).onError((_, _) {});
    await _auth.signOut();
    try {
      await _users.clearLocalData();
      await ref.read(draftStoreProvider).clear();
      // Hộp thư FCM nằm trên máy, không theo tài khoản: xoá để người đăng
      // nhập sau không đọc được thông báo cũ.
      await ref.read(messagingServiceProvider).clearInbox();
      ref.read(inboxRevisionProvider.notifier).bump();
    } on Object {
```

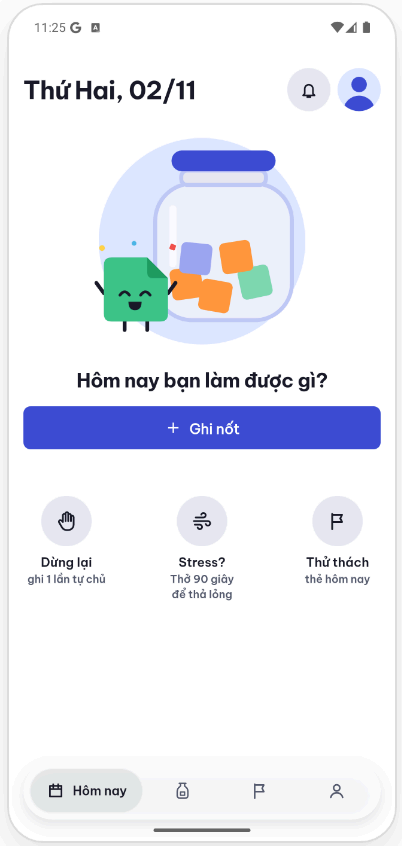
Real code on GitHub · FU-PRM-Projects/Lab-03 · branch main · key lines boxed in red

Authentication

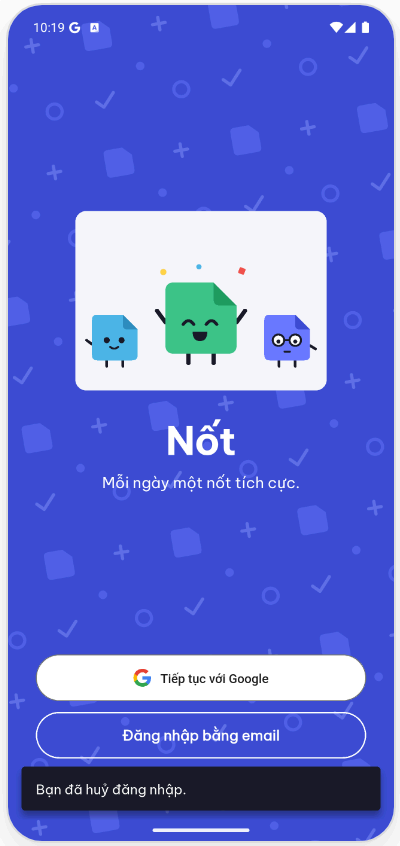
When it fails: cancel → “Bạn đã huỷ đăng nhập.” · offline → “Không có mạng. Kiểm tra Wi-Fi rồi thử lại.” · sign-out error → “Chưa đăng xuất được. Thử lại.”



Signed out → /welcome

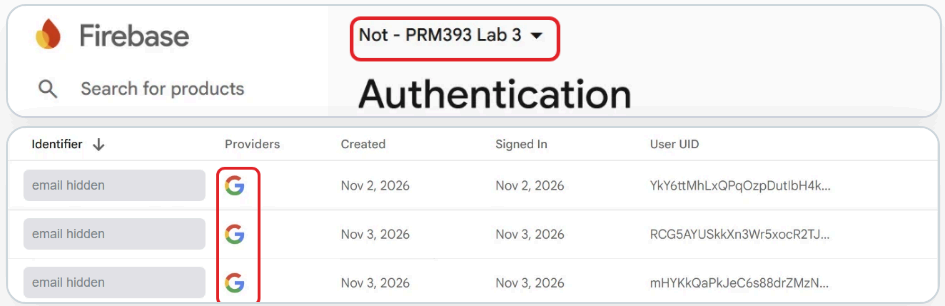


Signed in → /today



Cancelled → message

Console · Authentication › Users



Routes by auth state `lib/router/auth_redirect.dart`

```dart
final signedIn = auth.hasValue && auth.value != null;
if (!signedIn) {
  return onWelcome ? null : _withNext(AppRoutes.welcome, next);
}
if (onSplash || onWelcome) return next ?? AppRoutes.today;
```

Sign-out clears local data `auth_view_model.dart`

```dart
await _auth.signOut();
try {
  await _users.clearLocalData();
  await ref.read(draftStoreProvider).clear();
  // Hộp thư FCM nằm trên máy, không theo tài khoản: xoá để người đăng
  // nhập sau không đọc được thông báo cũ.
  await ref.read(messagingServiceProvider).clearInbox();
  ref.read(inboxRevisionProvider.notifier).bump();
```

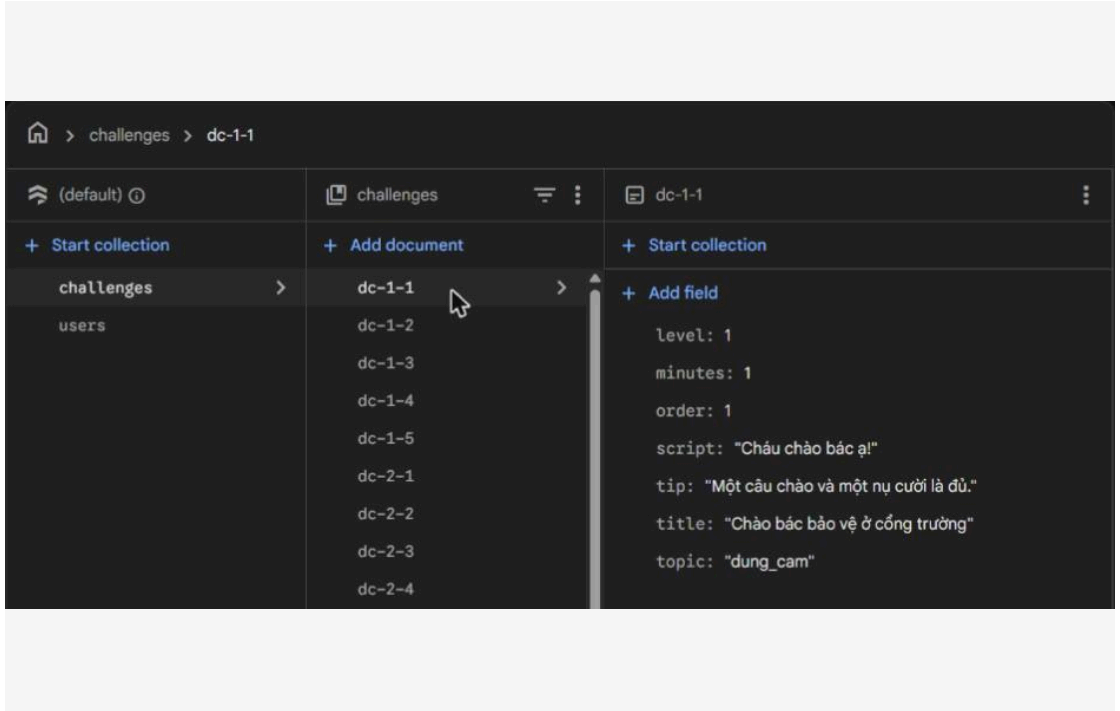
evidence/auth/ · lib/firebase/firebase_auth_service.dart

Cloud Firestore

Cloud Firestore is Firebase's NoSQL document database: collections → documents → fields, with live updates and an offline cache.

- `snapshots()` pushes every change to the app
- Writes wait on the device when offline, then sync
- Security Rules check each read and write on the server

In Nốt: private notes under users/{uid}/notes.



Documents in Firestore

Cloud Firestore

WHY WE USE IT

The Jar updates by itself

Today and the Jar watch a stream; no manual refresh.

Works offline

A note saved without Wi-Fi stays on the device (“Đã lưu trên máy”).

Owner-only data

Rules: only the owner, only valid notes; tested on the Emulator.

WHY NOT SOMETHING ELSE?

✕ Realtime Database

One big JSON tree, and a query can sort or filter on only one field.

✕ SQLite on the phone

Data stays on one device: no sync, lost with the phone.

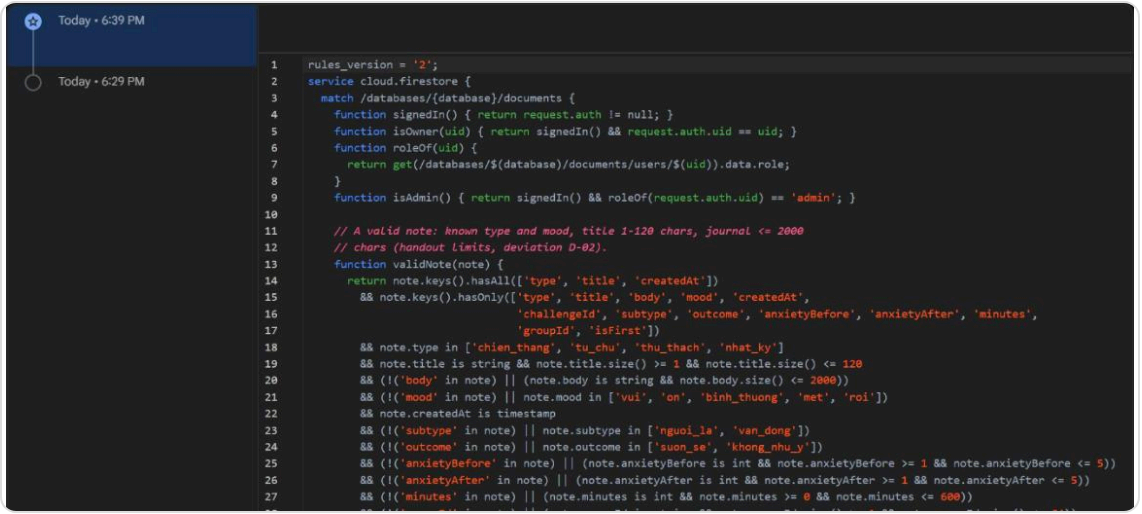
✕ Own REST API + SQL

We would host a server and write offline sync ourselves.

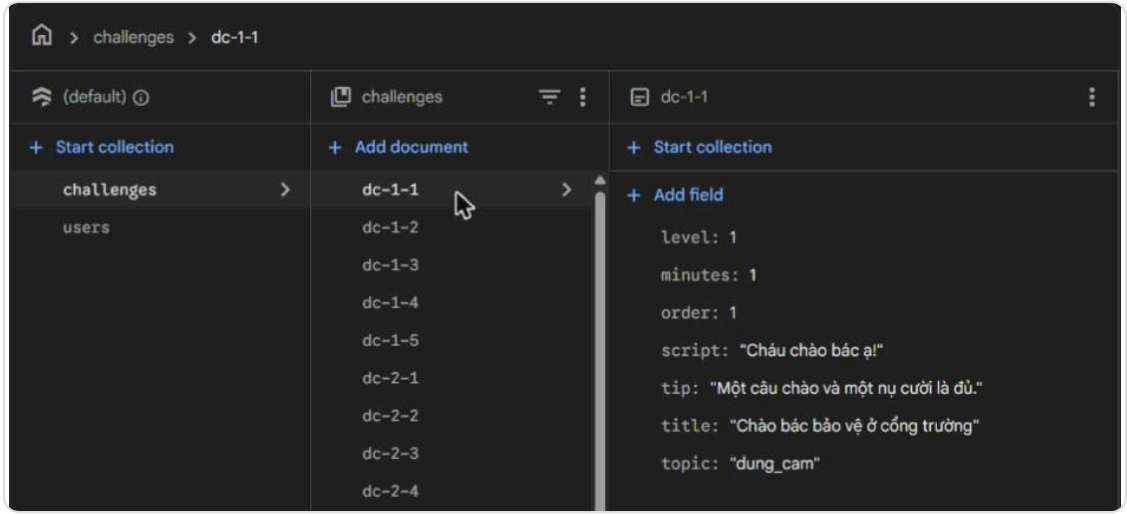
FPT Education
FPT UNIVERSITY

Cloud Firestore

Create the database, publish the rules from firebase/firestore.rules with `firebase deploy --only firestore`, and seed the shared challenge catalogue with `npm run seed`.



1 Firestore › Rules: our rules published from the repo (history on the left)



2 Firestore › Data: the seeded `challenges` collection

View demo and evidence →

Cloud Firestore

lib/firebase/firestore_note_repository.dart :180–192

Write a note with **doc.set**

```dart
@override
Future<Note> add(NoteDraft draft, {required DateTime createdAt}) =>
    _guard(() async {
      final doc = _notes().doc();
      final body = draft.body.trim();
      final pending = await _write(
        doc.set(<String, Object?>{
          'type': draft.type.id,
          'title': draft.title.trim(),
          'body': ?(body.isEmpty ? null : body),
          'mood': ?draft.mood?.id,
          'createdAt': Timestamp.fromDate(createdAt),
          'groupId': ?draft.groupId,
          'image': ?draft.image?.toMap(),
          if (draft.groupId != null && draft.isFirst) 'isFirst': true,
```

lib/firebase/firestore_note_repository.dart :87–95

Wait 2 s, else keep it on the device

```dart
/// còn nằm trên máy.
static Future<bool> _write(Future<void> write) async {
  try {
    await write.timeout(_syncWait);
    return false;
  } on TimeoutException {
    // Firestore vẫn giữ lệnh ghi và gửi lại khi có mạng.
    write.ignore();
    return true;
  }
}
```

lib/firebase/firestore_note_repository.dart :108–116

Live list from **snapshots()**

```dart
/// sang `AppException`.
Stream<List<Note>> _watch(Query<Map<String, Object?>> Function() query) {
  final Stream<QuerySnapshot<Map<String, Object?>>> snapshots;
  try {
    snapshots = query().snapshots(includeMetadataChanges: true);
  } on AppException catch (e, stack) {
    return Stream<List<Note>>.error(e, stack);
  }
  return snapshots
      .map(_toNotes)
      .transform(
```

firebase/firestore.rules :156–159

Only the owner, only valid notes

```
match /notes/{noteId} {
  allow read, delete: if isOwner(uid);
  allow create, update: if isOwner(uid) && validNote(request.resource.data)
}
```

Real code on GitHub · FU-PRM-Projects/Lab-03 · branch main · key lines boxed in red

Cloud Firestore

When it fails: offline → saved on the device + banner · load error or over 10 s → error state + Thử lại · denied → message, draft kept

Data model

`users/{uid}`	role, photo
`…/notes/{id}`	type, title, body, mood, createdAt, image
`…/groups/{id}`	first-time groups
`…/private/*`	habits, settings
`…/challengeProgress`	level per topic
`…/sharedStreaks`	shared streak image
`challenges/{id}`	catalogue, read-only

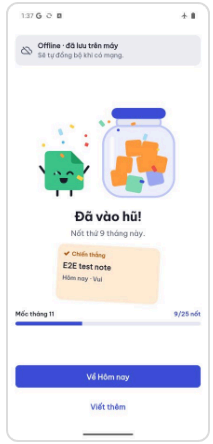
18 / 18 rules tests pass
Emulator · npm run test:rules · R-01 … R-18

Write a note `firestore_note_repository.dart`

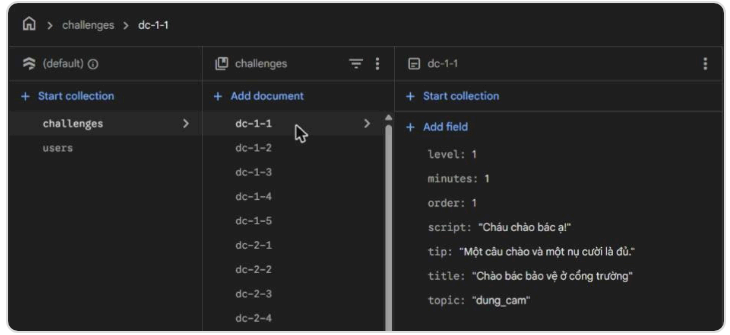
```
182         final doc = _notes().doc();
183         final body = draft.body.trim();
184         final pending = await _write(
185           doc.set(<String, Object?>{
186             'type': draft.type.id,
187             'title': draft.title.trim(),
188             'body': ?(body.isEmpty ? null : body),
189             'mood': ?draft.mood?.id,
190             'createdAt': Timestamp.fromDate(createdAt),
191             'groupId': ?draft.groupId,
192             'image': ?draft.image?.toMap(),
```

Only the owner, only valid notes `firestore.rules`

```
25    function validNote(note) {
26      return note.keys().hasAll(['type', 'title', 'createdAt'])
27        && note.keys().hasOnly(['type', 'title', 'body', 'mood', 'createdAt',
28                                'challengeId', 'subtype', 'outcome', 'anxietyBefor
29                                'groupId', 'isFirst', 'image'])
30        && note.type in ['chien_thang', 'tu_chu', 'thu_thach', 'nhat_ky']
31        && note.title is string && note.title.size() >= 1 && note.title.size() <=
32        && (!('body' in note) || (note.body is string && note.body.size() <= 2000)
33        && (!('mood' in note) || note.mood in ['vui', 'on', 'binh_thuong', 'met',
```



Offline: "Đã lưu trên máy"



Console · seeded challenges

firebase/firestore.rules · lib/firebase/firestore_note_repository.dart · evidence/firestore/ · evidence/deploy/

Cloudinary

Cloudinary is a cloud service that stores photos and serves them from an image CDN; the app posts a photo and gets back a URL and an ID.

- Unsigned upload preset: no API secret in the app
- Upload over HTTP (`dio`) with progress and cancel
- Photos served over HTTPS from `res.cloudinary.com`

In Nốt: profile photos, note attachments and group covers.

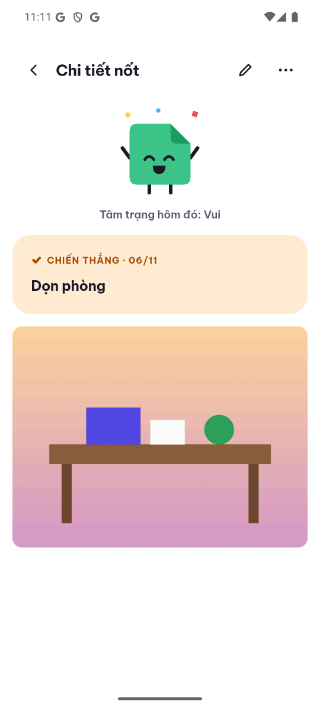


Photo displayed in a saved note

Cloudinary

WHY WE USE IT

Free plan with an image CDN

Photos load fast over HTTPS from `res.cloudinary.com` at no cost.

Progress and cancel

The user sees the upload percentage and can stop it.

Only a reference in Firestore

`{url, publicId}`, checked by `validImage`.

WHY NOT SOMETHING ELSE?

✕ Firebase Storage

A new bucket needs the paid Blaze plan; our project is on Spark.

✕ Photo as base64 in Firestore

A document holds at most 1 MiB, and every read downloads the photo.

✕ Amazon S3

Uploads need a signed request, so the secret would need a backend.

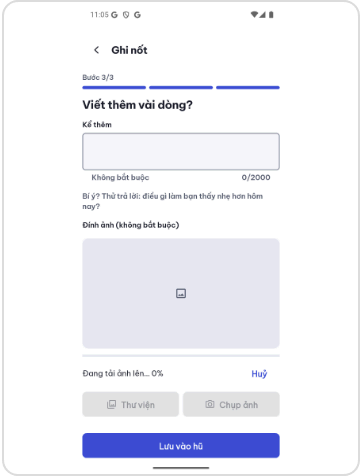
FPT Education
FPT UNIVERSITY

Cloudinary

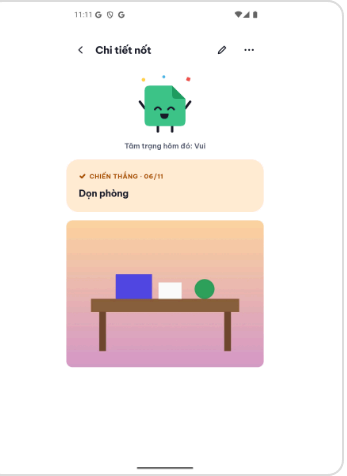
On Cloudinary, create an unsigned upload preset `PRM_Lab3` with type and size limits. The app gets the cloud name and preset through --dart-define, then uploads straight to Cloudinary.

```
126          ),
127          imageServiceProvider.overrideWithValue(
128            CloudinaryImageService(
129              currentUid: uid,
130              cloudName: _cloudinaryCloudName,
131              uploadPreset: _cloudinaryUploadPreset,
132            ),
133          ),
134          // Không override thì dùng bản Noop: lưu avatar "thành công" nhưng không
```

1 Cloud name + unsigned preset `PRM_Lab3` passed to the app



2 Upload to Cloudinary with progress and cancel



3 Photo served back from `res.cloudinary.com`

View demo and evidence →

Cloudinary

lib/firebase/cloudinary_image_service.dart :72–82

Unsigned preset, random id, upload with progress

```
71          }
72          final form = FormData.fromMap(<String, Object>{
73            'file': MultipartFile.fromBytes(image.bytes, filename: image.fileName),
74            'upload_preset': uploadPreset,
75            // public_id ngẫu nhiên: không đoán được, không ghi đè được.
76            'public_id': 'not/${folder.id}/${_randomId()}',
77          });
78          final response = await _dio.post<Map<String, dynamic>>(
79            'https://api.cloudinary.com/v1_1/$cloudName/image/upload',
80            data: form,
81            cancelToken: cancelToken,
82            onSendProgress: (sent, total) {
83              if (total > 0 && !out.isClosed) {
```

lib/firebase/cloudinary_image_service.dart :89–101

Read **secure_url** and **public_id**

```
88          );
89          final data = response.data;
90          final url = data?['secure_url'];
91          final publicId = data?['public_id'];
92          if (url is! String || publicId is! String) {
93            throw const AppException(
94              AppErrorKind.unknown,
95              code: 'cloudinary-bad-response',
96            );
97          }
98          out.add(
99            ImageUploadProgress(
100             1,
101             image: ImageRef(url: url, publicId: publicId),
102           ),
```

firebase/firestore.rules :20–27

validImage: only a Cloudinary link

```
19        // Firestore only keeps the reference.
20        function validImage(image) {
21          return image is map
22            && image.keys().hasOnly(['url', 'publicId'])
23            && image.keys().hasAll(['url', 'publicId'])
24            && image.url is string && image.url.size() <= 500
25            && image.url.matches('https://res[.]cloudinary[.]com/.+')
26            && image.publicId is string && image.publicId.size() >= 1 && image.publicI
27        }
28
```

lib/main.dart :127–133

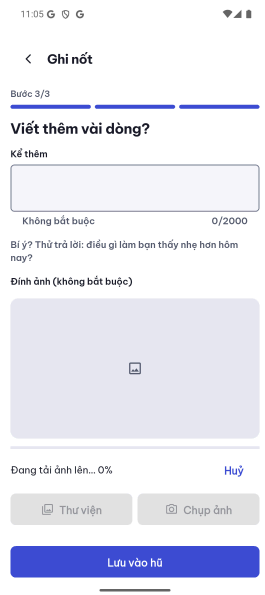
Cloud name and preset from --dart-define

```
126           ),
127           imageServiceProvider.overrideWithValue(
128             CloudinaryImageService(
129               currentUid: uid,
130               cloudName: _cloudinaryCloudName,
131               uploadPreset: _cloudinaryUploadPreset,
132             ),
133           ),
134           // Không override thì dùng bản Noop: lưu avatar "thành công" nhưng không
```

Real code on GitHub · FU-PRM-Projects/Lab-03 · branch main · key lines boxed in red

Photos on Cloudinary

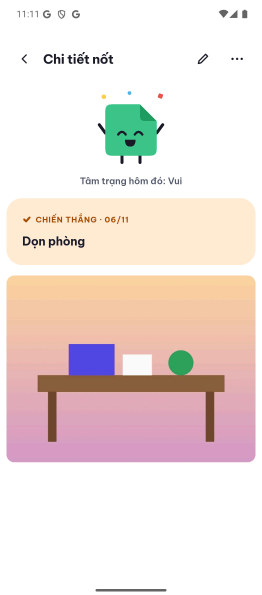
Why: Firebase Storage needs the paid Blaze plan, so profile, note and group photos are uploaded to Cloudinary



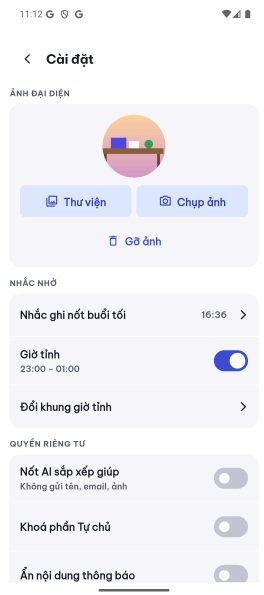
1 · Uploading, with progress



2 · Attached to the note



3 · Saved note with its photo



4 · Profile photo in Settings

Upload · unsigned preset, random id `cloudinary_image_service.dart`

```
73          'file': MultipartFile.fromBytes(image.bytes, filename: image.fileName),
74          'upload_preset': uploadPreset,
75          // public_id ngẫu nhiên: không đoán được, không ghi đè được.
76          'public_id': 'not/${folder.id}/${_randomId()}',
77        });
78        final response = await _dio.post<Map<String, dynamic>>(
79          'https://api.cloudinary.com/v1_1/$cloudName/image/upload',
80          data: form,
81          cancelToken: cancelToken,
82          onSendProgress: (sent, total) {
83            if (total > 0 && !out.isClosed) {
84              // Dành 5% cuối cho lúc server xử lý.
85              out.add(ImageUploadProgress(sent / total * 0.95));
86            }
```

- **No secret in the app:** only the cloud name and an unsigned preset
- **Firestore keeps the link only:** `{url, publicId}`, checked by `validImage` in the rules
- **When it fails:** no Wi-Fi, expired session, wrong type or over 5 MB each show a message
- **Weekly card PDF:** saved locally in the current build

lib/firebase/cloudinary_image_service.dart · firebase/firestore.rules · evidence/cloudinary/

FPT Education
FPT UNIVERSITY

Firebase Cloud Messaging

Firebase Cloud Messaging (FCM) is Google's free push service: a message sent to a device token reaches the app whether it is open, in the background or closed.

- Each install has a device token
- A message carries a `data` part, here `route`
- Android 13+ needs `POST_NOTIFICATIONS`

In Nốt: tapping a test push can open the Jar.



Push received in the foreground

FPT Education
FPT UNIVERSITY

Firebase Cloud Messaging

WHY WE USE IT

The success criterion starts here

Log a win in under 30 s after tapping a notification.

Open the right screen

`data.route = /jar` opens the Jar; only allow-listed routes.

All three app states

Foreground, background and terminated, as Lab 3 requires.

WHY NOT SOMETHING ELSE?

✕ OneSignal

Another SDK and account; on Android it delivers through FCM anyway.

✕ Local notifications only

Scheduled on the phone, cannot be sent from a server (we still use them for the evening reminder).

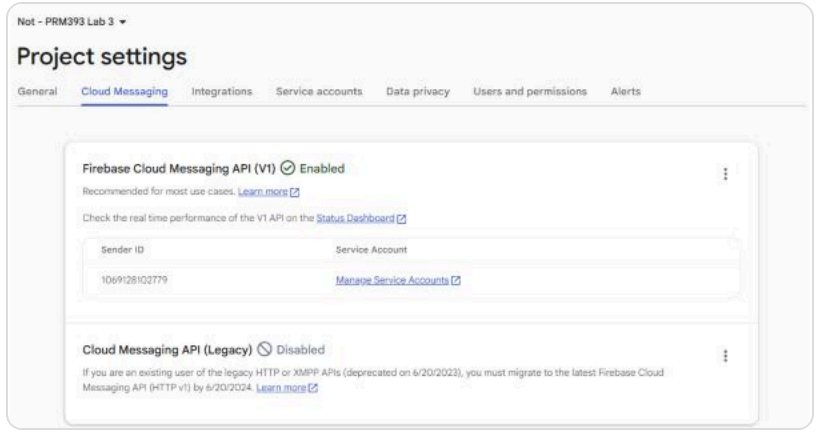
✕ Polling a server

The app would wake up again and again and drain the battery.

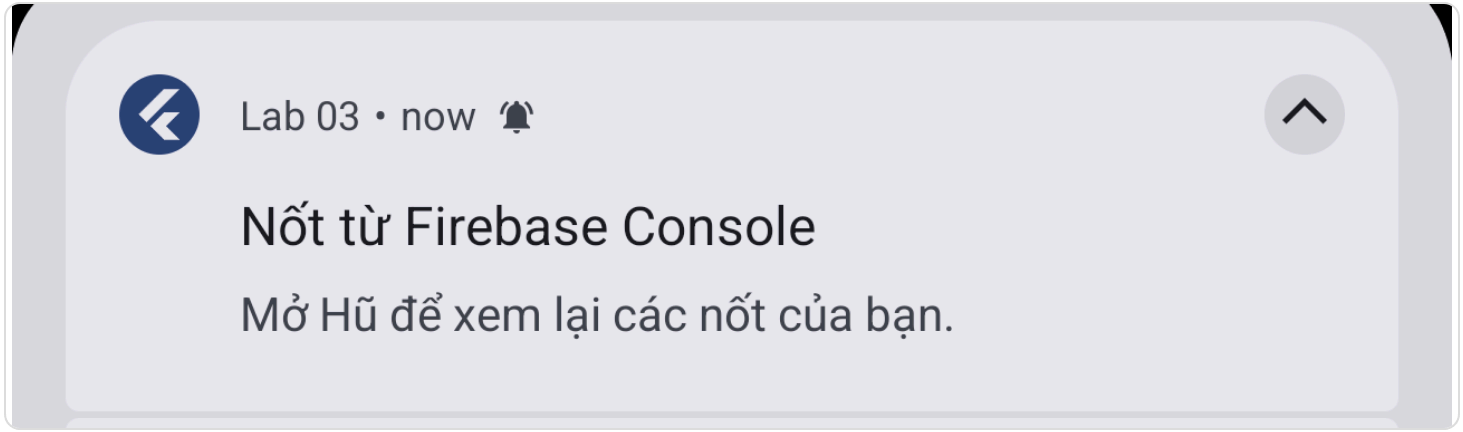
FPT Education
FPT UNIVERSITY

Firebase Cloud Messaging

Check that the Cloud Messaging API (V1) is on, then send a test message from the Console to the device token printed in the debug log, with data route=/jar.



1 Project settings › Cloud Messaging: API (V1) enabled



2 Messaging › Send test message to the device token: it arrives on the phone

View demo and evidence →

Firebase Cloud Messaging

lib/main.dart :82–86

Background handler, start messaging

```
81  installCrashHandlers(enabled: crashEnabled);
82  final messaging = FirebaseMessagingService();
83  if (!kIsWeb) {
84    FirebaseMessaging.onBackgroundMessage(firebaseMessagingBackgroundHandler);
85    unawaited(messaging.start());
86  }
87  final auth = FirebaseAuthService(
```

lib/firebase/firebase_messaging_service.dart :77–87

Foreground, background and terminated

```
76        );
77        FirebaseMessaging.onMessage.listen(showForeground);
78        FirebaseMessaging.onMessageOpenedApp.listen((m) async {
79          await _record(m, MessageOrigin.background);
80          _routes.add(routeFromData(m.data));
81        });
82        final initial = await _messaging.getInitialMessage();
83        if (initial != null) {
84          await _record(initial, MessageOrigin.terminated);
85          _pendingRoute = routeFromData(initial.data);
86          _routes.add(_pendingRoute!);
87        }
88        unawaited(_logToken());
```

lib/firebase/firebase_messaging_service.dart :97–107

Foreground: local notification + route

```
96  @visibleForTesting
97  Future<void> showForeground(RemoteMessage m) async {
98    await _record(m, MessageOrigin.foreground);
99    final n = m.notification;
100   if (n != null) {
101     await _local.show(
102       id: m.hashCode,
103       title: n.title,
104       body: n.body,
105       notificationDetails: const NotificationDetails(android: _channel),
106       payload: routeFromData(m.data),
107     );
108   }
```

lib/router/app_router.dart :234–237

Tapped route → open the screen

```
233 // authRedirect đưa về Welcome).
234 final messaging = ref.read(messagingServiceProvider);
235 final routeSub = messaging.openedRoutes.listen(router.go);
236 final pending = messaging.takePendingRoute();
237 if (pending != null) router.go(pending);
238
```

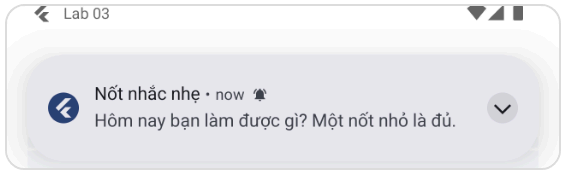
Real code on GitHub · FU-PRM-Projects/Lab-03 · branch main · key lines boxed in red

Cloud Messaging: requirements met

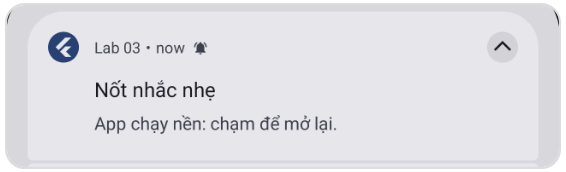
When it fails: permission denied → dialog + “Mở cài đặt” · unknown route → opens Today · FCM start error → logged, the app keeps running

Foreground · background · terminated

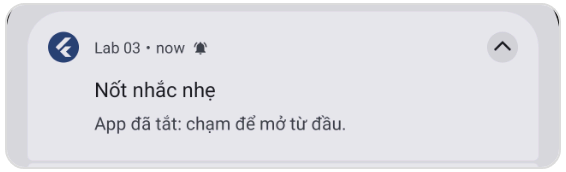
Foreground



Background

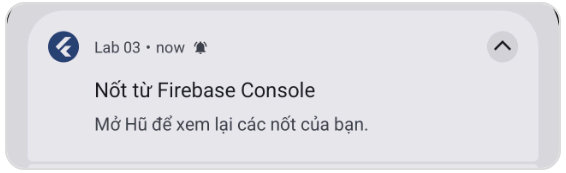


Terminated

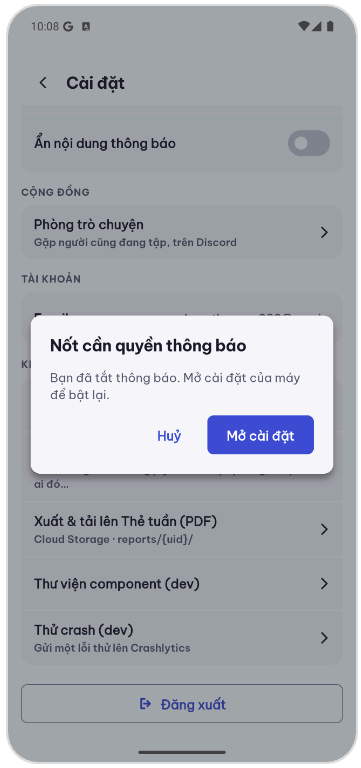


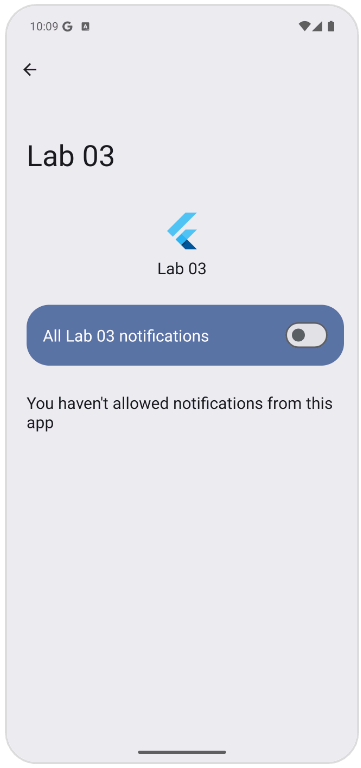
Test message from the Console

Console



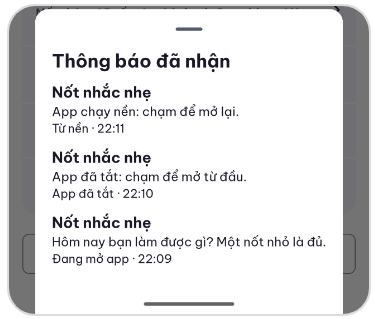
Asks permission (Android 13+)



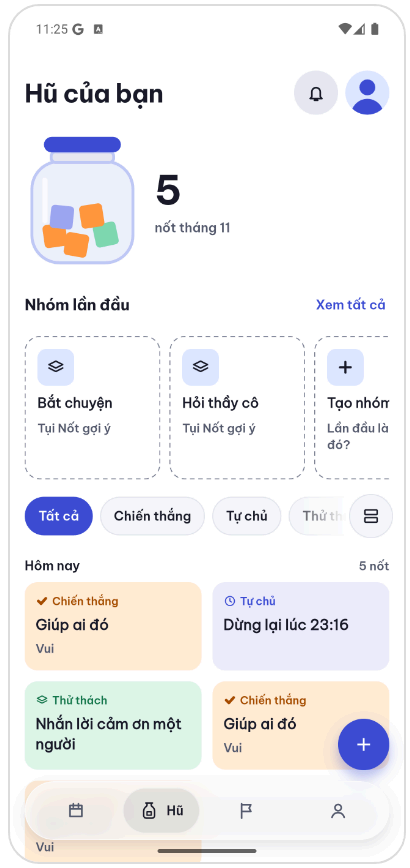


Denied → “Mở cài đặt” → Android settings

List of received notifications



Tap opens the screen



data route = /jar → Jar tab

evidence/fcm/ · lib/firebase/firebase_messaging_service.dart

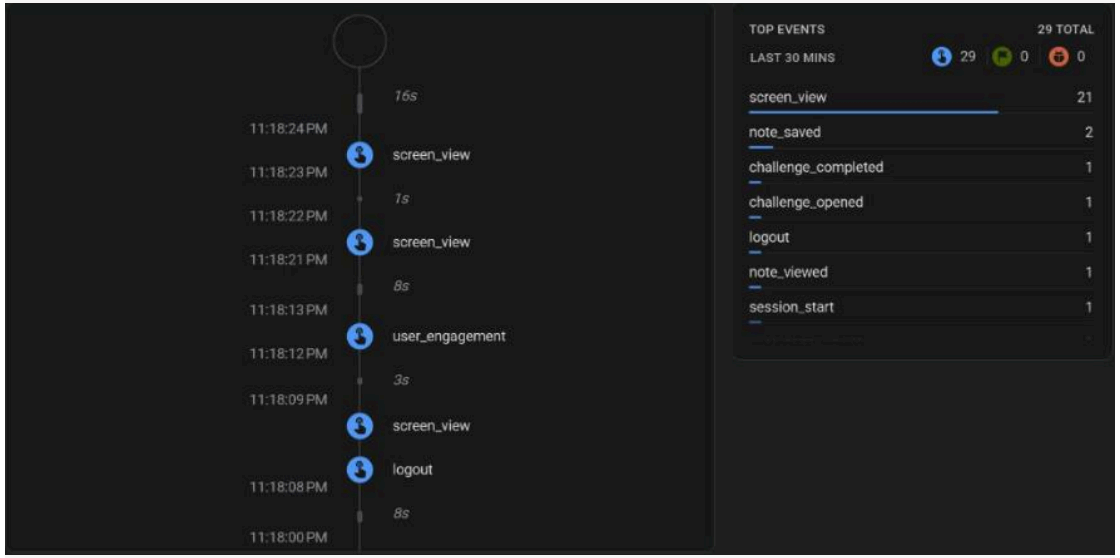
FPT Education
FPT UNIVERSITY

Google Analytics for Firebase

Google Analytics for Firebase records the events the app logs, each with parameters, and shows them in reports and live in DebugView.

- `logEvent(name, parameters)` for custom events
- `setUserId(uid)` after sign-in
- DebugView shows a debug device within seconds

In Nốt: note_saved records duration_ms.



Events arriving in DebugView

FPT Education
FPT UNIVERSITY

Google Analytics for Firebase

WHY WE USE IT

Measure the 30-second goal

`note_saved.duration_ms`: 7.7 s in our run.

No personal data

No note text, titles, emails or names are logged.

Never breaks a feature

`track()` catches a failed log; the action still works.

WHY NOT SOMETHING ELSE?

✕ Mixpanel / Amplitude

Another SDK and account, with usage limits on the free plan.

✕ Events as Firestore documents

A write per event, and no reports or DebugView.

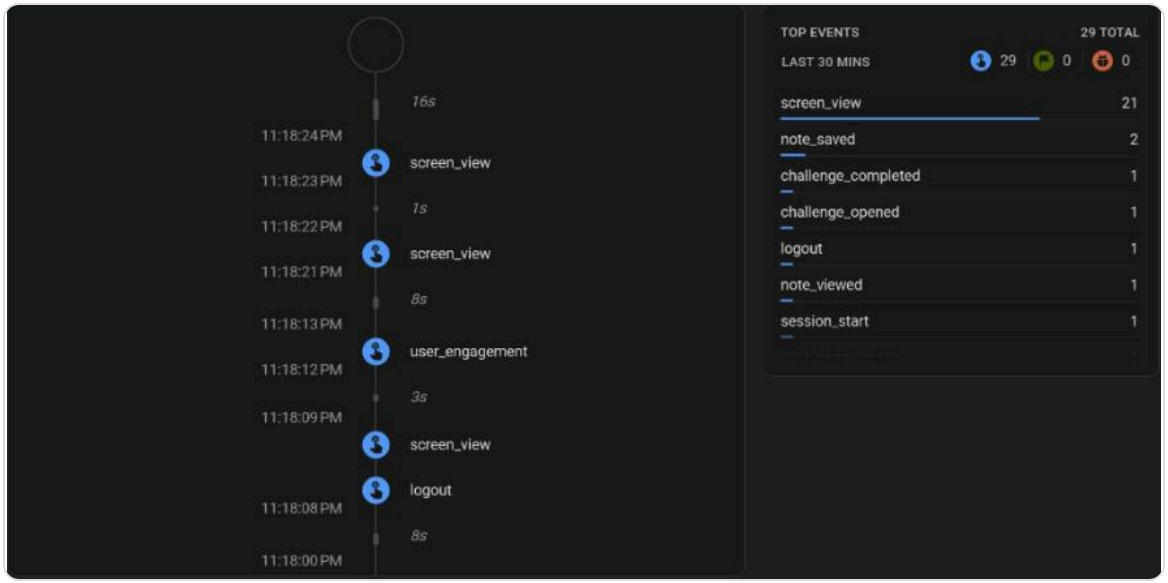
✕ No analytics

We could not check the 30-second goal.

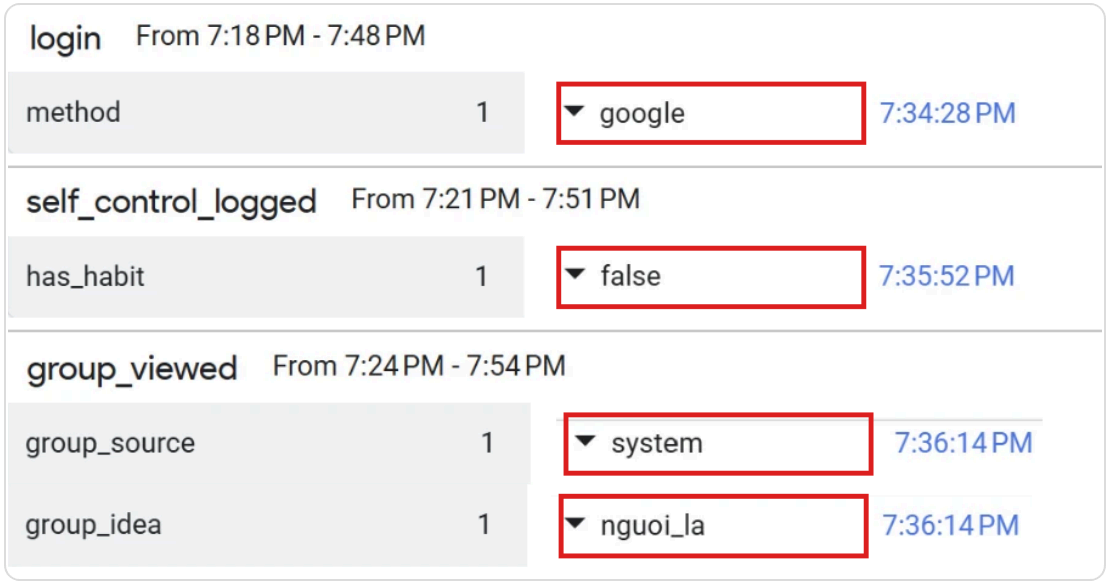
FPT Education
FPT UNIVERSITY

Google Analytics for Firebase

Analytics is linked when the project is created. Mark the emulator as a debug device with `adb shell setprop debug.firebase.analytics.app com.quanphungg.lab03`, use the app, and watch DebugView.



1 Analytics › DebugView: events from the debug device



2 DebugView: open an event to check its parameters

View demo and evidence →

Google Analytics for Firebase

lib/viewmodels/note_composer_view_model.dart :253–264
note_saved with **duration_ms**

```
252         }
253         unawaited(
254           ref.read(analyticsServiceProvider).track(
255             AnalyticsEvents.noteSaved,
256             <String, Object>{
257               AnalyticsParams.mood: toSave.mood?.name ?? 'none',
258               AnalyticsParams.noteType: 'chien_thang',
259               AnalyticsParams.hasJournal: '${toSave.body.isNotEmpty}',
260               AnalyticsParams.durationMs: ref
261                   .read(clockProvider)
262                   .now()
263                   .difference(_openedAt)
264                   .inMilliseconds,
265             },
```

lib/services/analytics_service.dart :39–46
track() never breaks the user's action

```
38    /// Ghi sự kiện mà không bao giờ làm hỏng luồng chính của người dùng.
39    extension SafeAnalytics on AnalyticsService {
40      Future<void> track(String name, [Map<String, Object>? parameters]) async {
41        try {
42          await logEvent(name, parameters);
43        } on Object catch (error) {
44          debugPrint('[analytics] bỏ qua $name: $error');
45        }
46      }
47    }
```

lib/firebase/firebase_analytics_service.dart :11–20
Forward to Firebase Analytics

```
10
11    @override
12    Future<void> logEvent(String name, [Map<String, Object>? parameters]) =>
13        _analytics.logEvent(name: name, parameters: parameters);
14
15    @override
16    Future<void> logScreen(String screenName) =>
17        _analytics.logScreenView(screenName: screenName);
18
19    @override
20    Future<void> setUserId(String? uid) => _analytics.setUserId(id: uid);
21  }
```

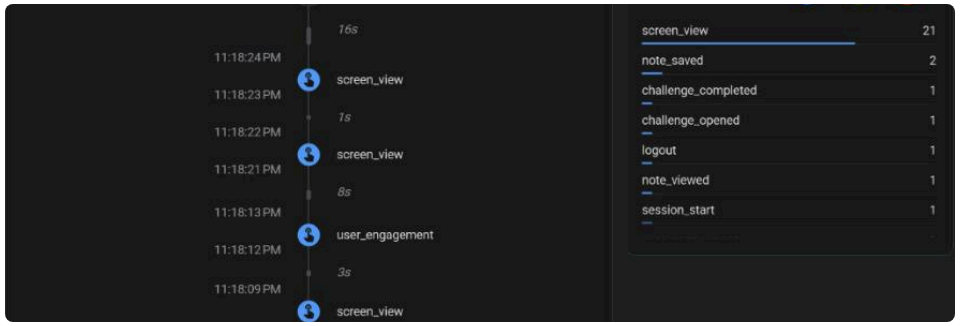
lib/services/analytics_events.dart :5–13
Every event name in one place

```
4   /// dung nốt, email hay tên nhóm.
5   abstract final class AnalyticsEvents {
6     static const String login = 'login';
7     static const String logout = 'logout';
8     static const String noteSaved = 'note_saved';
9     static const String challengeCompleted = 'challenge_completed';
10    static const String reportExported = 'report_exported';
11    static const String challengeOpened = 'challenge_opened';
12    static const String noteViewed = 'note_viewed';
13    static const String groupViewed = 'group_viewed';
14    static const String selfControlLogged = 'self_control_logged';
```

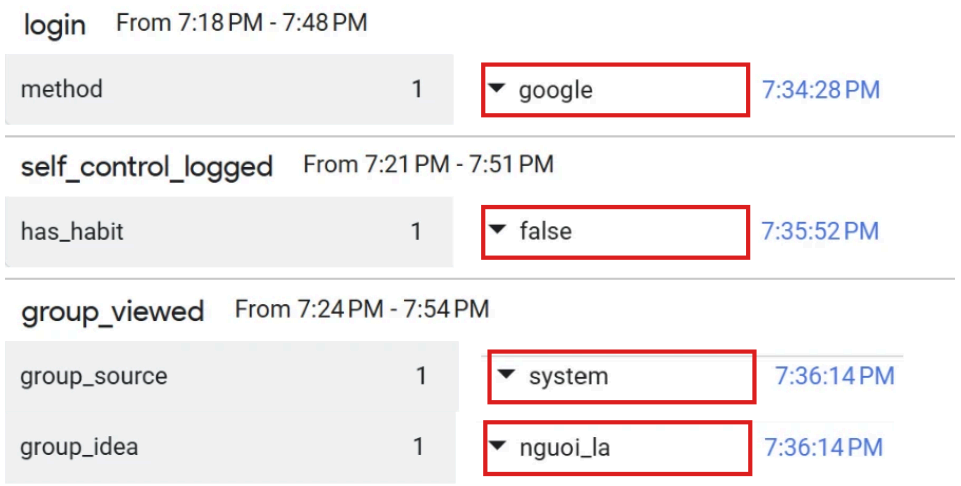
Real code on GitHub · FU-PRM-Projects/Lab-03 · branch main · key lines boxed in red

Analytics

When it fails: a failed log is caught and skipped (SafeAnalytics.track), so the user's action never fails



1 · DebugView · top events of one run



2 · Values sent, cut from DebugView panels

Nine custom events go through an `AnalyticsService` interface (Firebase in the app, a console logger on the Emulator). No titles, note text, emails or names are logged. The Firebase user ID is set after sign-in and cleared on sign-out.

- **login / logout** · `method`
- **note_saved** · `mood, note_type, has_journal, duration_ms`. Tracks the 30-second goal: 7.7 s in this run.
- **note_viewed / group_viewed** · `note_type` · `group_source, group_idea, note_count`
- **challenge_opened / challenge_completed** · `challenge_id, note_type`
- **self_control_logged** · `has_habit`
- **report_exported** · `format`. Fires after a successful upload, so it needs the Storage bucket.

Checked three ways: DebugView (left), `adb logcat` for all events except report_exported, and unit tests that assert the parameters.

docs/analytics-plan.md §4 event table · evidence/analytics/

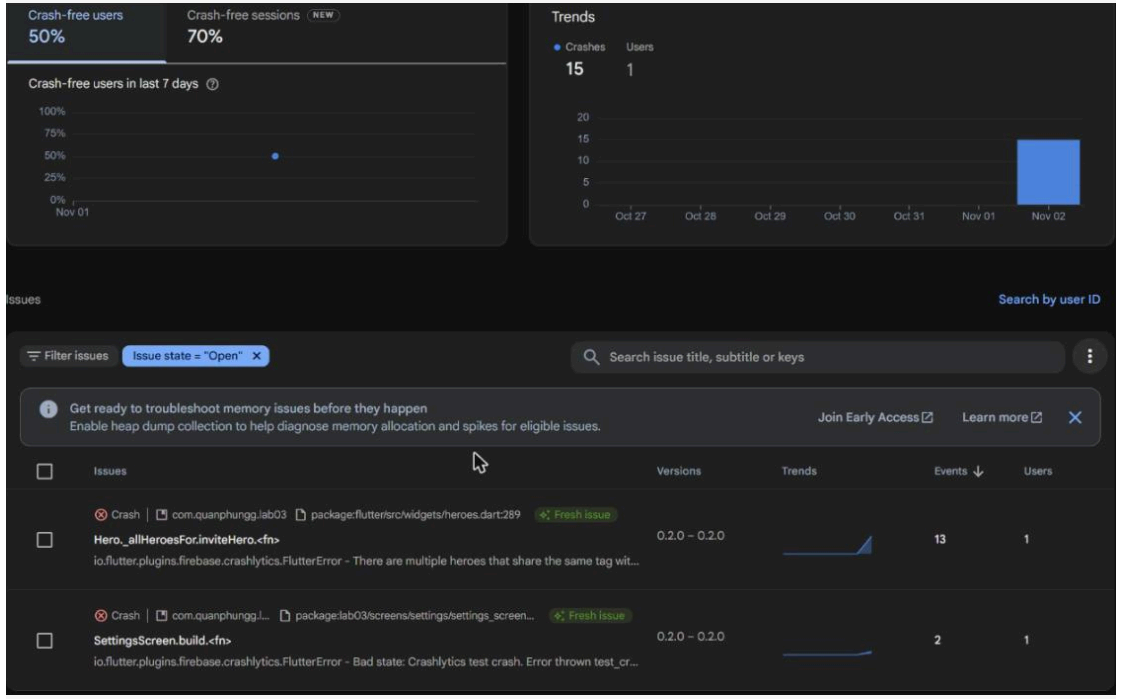
FPT Education
FPT UNIVERSITY

Firebase Crashlytics

Firebase Crashlytics collects crashes and caught errors from real devices and groups them into issues with the stack trace, custom keys and user ID.

- Fatal: uncaught errors, sent on the next launch
- Non-fatal: `recordError` for errors the app caught
- Context: user ID and custom keys

In Nốt: a failed note save reports note_save.



Crashlytics issues dashboard

FPT Education
FPT UNIVERSITY

Firebase Crashlytics

WHY WE USE IT

Real devices, not the Emulator
The Emulator Suite has no Crashlytics.

Caught errors still count
The user sees a message; we still get the report.

It found a real bug
Two add-note buttons shared a Hero tag; fixed.

WHY NOT SOMETHING ELSE?

✕ **Sentry**
Another account and SDK, with a monthly error quota on the free plan.

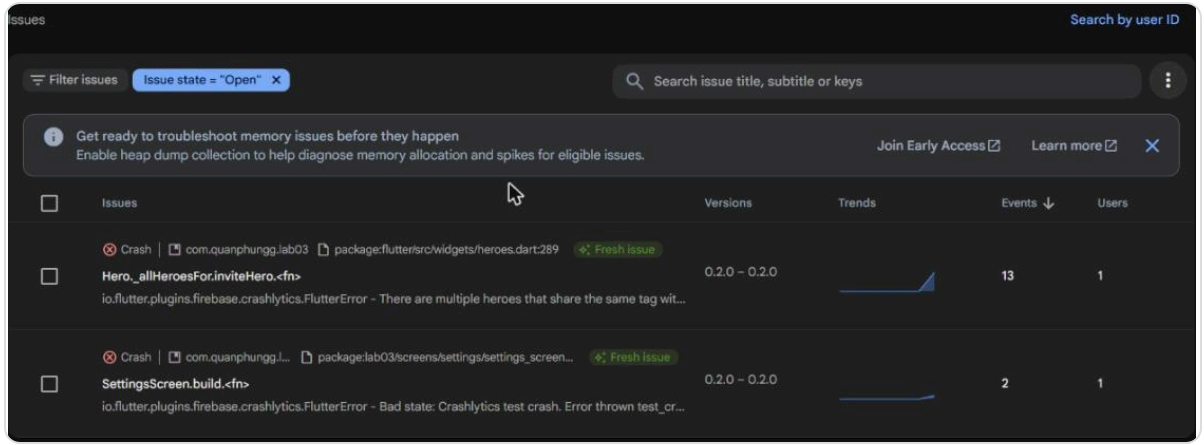
✕ **Only `print` / logcat**
Works on our emulator, not on a user's phone after release.

✕ **Error rows in Firestore**
No grouping and no stack trace view; a write per error.

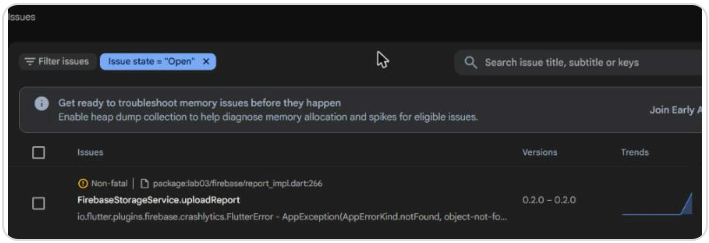
FPT Education
FPT UNIVERSITY

Firebase Crashlytics

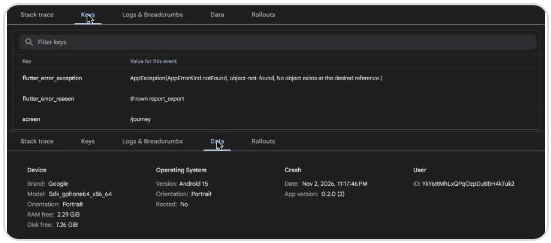
Add the Crashlytics Gradle plugin, send a test crash from Settings › “Thử crash (dev)” to activate the dashboard, then read non-fatals with their keys and user ID.



1 Crashlytics › Issues: the test crash and a real Hero error



2 Issues: a non-fatal error the app reported



3 Issue › Keys: `screen` and the user ID

[View demo and evidence →]

Firebase Crashlytics

lib/firebase/crash_handlers.dart :9–19

Uncaught errors → Crashlytics

```dart
/// lên dashboard thật.
void installCrashHandlers({required bool enabled}) {
  if (!enabled) return;

  // Lỗi đồng bộ trong framework (build, onPressed, ...).
  FlutterError.onError = FirebaseCrashlytics.instance.recordFlutterFatalError;

  // Lỗi async không đi qua framework.
  PlatformDispatcher.instance.onError = (error, stack) {
    FirebaseCrashlytics.instance.recordError(error, stack, fatal: true);
    return true;
  };
}
```

lib/firebase/firebase_crash_reporter.dart :9–20

Report a caught error (non-fatal)

```dart

  @override
  Future<void> recordError(
    Object error,
    StackTrace? stack, {
    String? reason,
    bool fatal = false,
  }) => FirebaseCrashlytics.instance.recordError(
    error,
    stack,
    reason: reason,
    fatal: fatal,
  );

```

lib/viewmodels/note_composer_view_model.dart :276–281

A failed save → non-fatal **note_save**

```dart
      state = state.copyWith(saving: false, savedNoteId: note.id);
    } on Object catch (error) {
      unawaited(
        ref
            .read(crashReporterProvider)
            .recordError(error, null, reason: 'note_save'),
      );
      try {
```

lib/main.dart :80–81

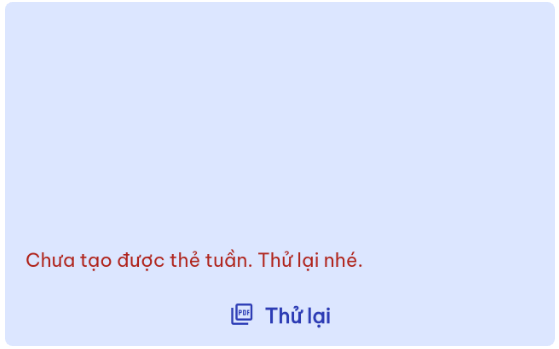
Only on the real project, not on web

```dart
  // Android/iOS với project thật.
  const crashEnabled = !useFirebaseEmulator && !kIsWeb;
  installCrashHandlers(enabled: crashEnabled);
  final messaging = FirebaseMessagingService();
```

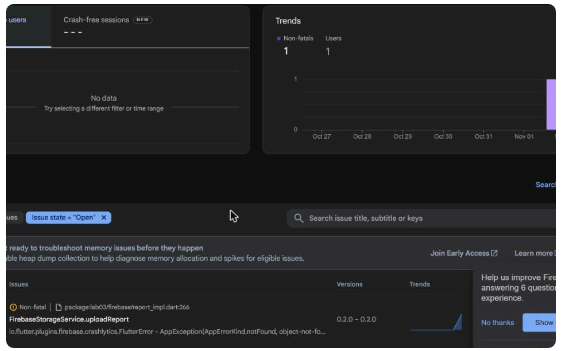
Real code on GitHub · FU-PRM-Projects/Lab-03 · branch main · key lines boxed in red

Crashlytics

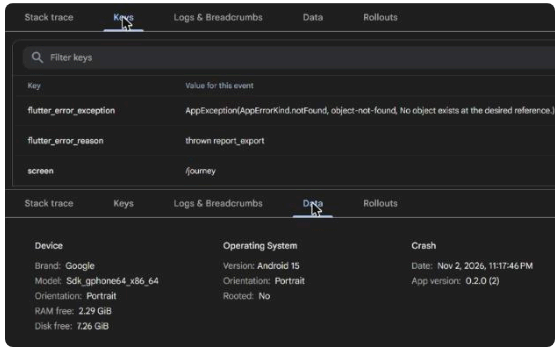
When it fails: the user still sees the error message; Emulator and web builds use a no-op reporter



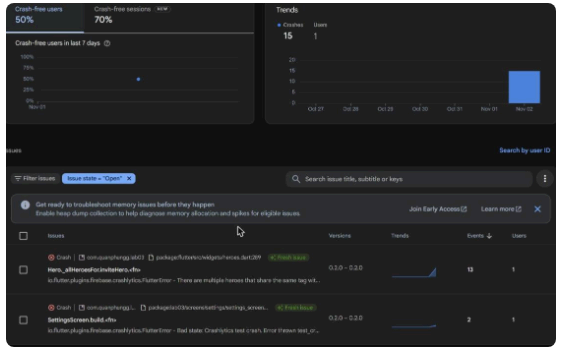
1 · User still sees a message



2 · Non-fatal in Console



3 · Key screen · user ID



4 · Test crash + a real Hero error

Crashlytics collects errors from real devices, so we see problems the Emulator never shows. Two kinds reach the dashboard:

- **Non-fatal**: the app caught the error and kept running. Exporting the weekly card fails because the Storage bucket is not provisioned; the user sees "Chưa tạo được thẻ tuần. Thử lại nhé." and the app records the error (1, 2).
- **Context on every report**: the Firebase user ID and a custom key `screen` (here `/journey`) show who hit it and where (3).
- **Crash**: a dev-only "Thử crash" button sends a test crash. The same dashboard also surfaced a real Hero error, caused by two add-note buttons sharing one tag. We fixed it with `heroTag: null` (4).

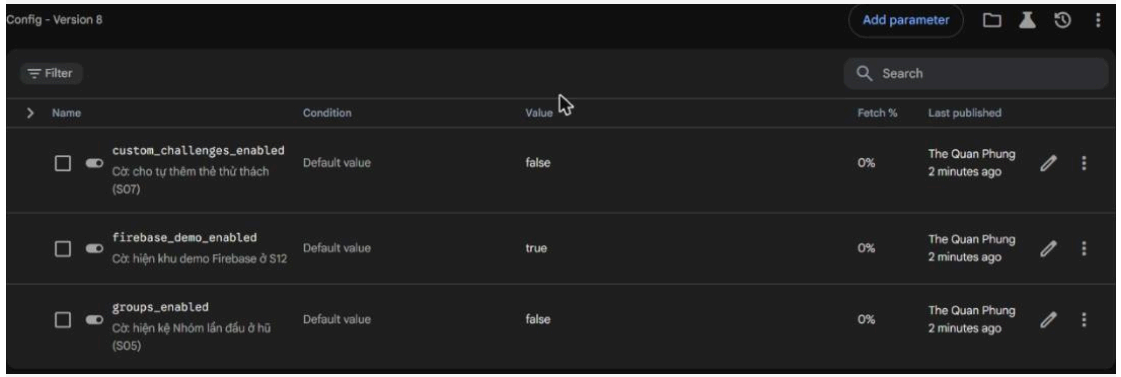
docs/analytics-plan.md §6 · evidence/crashlytics/

Firebase Remote Config

Firebase Remote Config is a set of key–value parameters on Firebase that the app fetches at run time, so values change from the Console without a new build.

- `setDefaults`: in-app values for first launch and errors
- `fetchAndActivate`: download and apply server values
- `minimumFetchInterval`: how long values are cached

In Nốt: groups_enabled toggles the Jar's groups shelf.



Feature flags in Firebase Console

Firebase Remote Config

WHY WE USE IT

Turn a feature off fast

Same APK before and after; only the Console value changed.

Safe when it fails

Every flag defaults to on; a failed fetch keeps features.

Values live in the repo

A template file, deployed with the Firebase CLI.

WHY NOT SOMETHING ELSE?

✕ Hard-coded flags

Every change needs a new build and a new install.

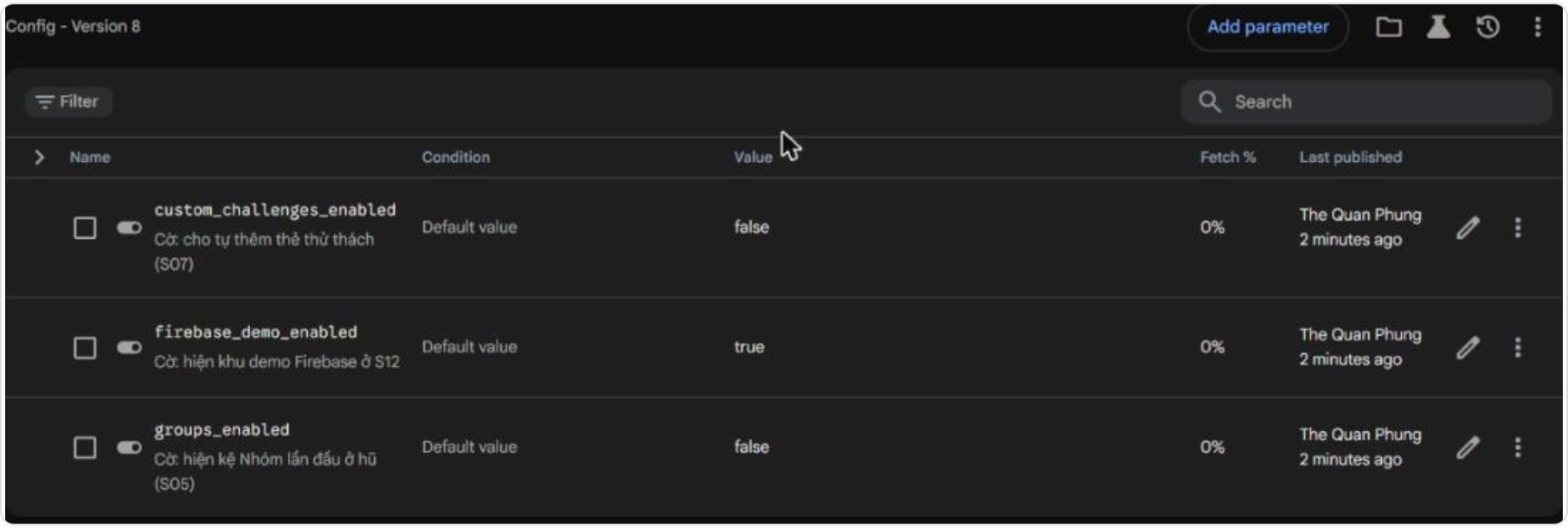
✕ A flags document in Firestore

No built-in defaults or cache; a read on every launch.

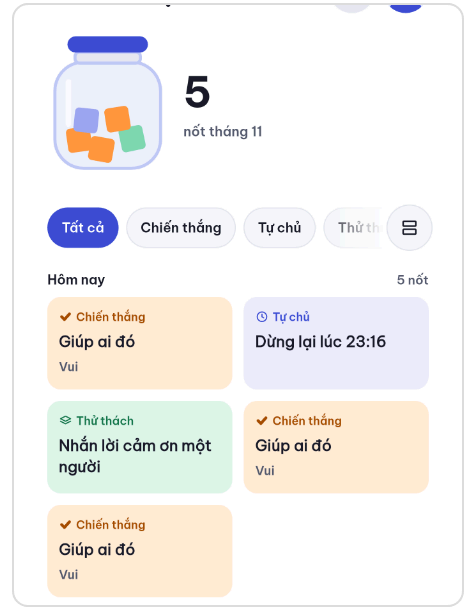
FPT Education
FPT UNIVERSITY

Firebase Remote Config

Publish firebase/remoteconfig.template.json with `firebase deploy --only remoteconfig`, then flip a flag on the Console and reopen the app: no new build.



1 Remote Config: the feature flags



2 Flag set to false → the groups shelf is gone

View demo and evidence →

Firebase Remote Config

lib/firebase/firebase_remote_config_service.dart :53–58
In-app defaults: every flag on

```
52        final service = FirebaseRemoteConfigService._(config);
53        try {
54          await config.setDefaults(<String, Object>{
55            'quick_win_chips': jsonEncode(InAppRemoteConfig.defaultQuickWinChips),
56            'groups_enabled': true,
57            'custom_challenges_enabled': true,
58            'challenge_submissions_enabled': true,
59            'discord_enabled': true,
```

lib/firebase/firebase_remote_config_service.dart :75–87
Cache 1 h in release, then fetch

```
74          });
75          await config.setConfigSettings(
76            RemoteConfigSettings(
77              fetchTimeout: const Duration(seconds: 10),
78              // Debug lấy ngay để demo "đổi trên Console, không build lại".
79              minimumFetchInterval: kReleaseMode
80                  ? const Duration(hours: 1)
81                  : Duration.zero,
82            ),
83          );
84          await config.fetchAndActivate();
85        } on Object catch (error) {
86          debugPrint('[remote-config] dùng giá trị mặc định: $error');
87        }
88        return service;
```

lib/screens/groups/group_shelf.dart :33–36
groups_enabled hides the shelf

```
32      Widget build(BuildContext context, WidgetRef ref) {
33        // Cờ Remote Config `groups_enabled`: tắt thì ẩn cả kệ.
34        if (!ref.watch(remoteConfigServiceProvider).groupsEnabled) {
35          return const SizedBox.shrink();
36        }
37        final state = ref.watch(groupsViewModelProvider);
```

lib/screens/challenges/challenges_screen.dart :150–152
custom_challenges_enabled

```
149
150       // Remote Config `custom_challenges_enabled`: tắt thì ẩn nút tự thêm thẻ.
151       final remoteConfig = ref.watch(remoteConfigServiceProvider);
152       final canAdd = remoteConfig.customChallengesEnabled;
153       // Remote Config `challenge_submissions_enabled`: tắt thì ẩn nút gửi thẻ
```

Real code on GitHub · FU-PRM-Projects/Lab-03 · branch main · key lines boxed in red

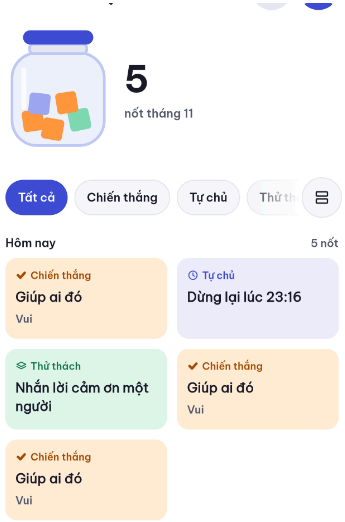
Remote Config

When it fails: fetch error → in-app defaults · broken JSON on the Console → default list · refresh shows “Cấu hình chưa có gì mới.”

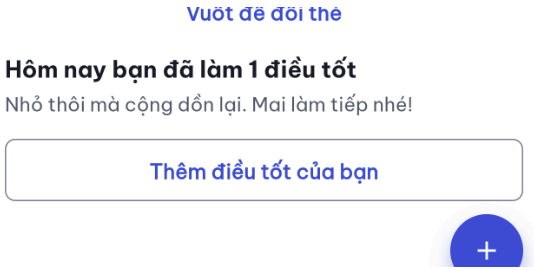
Same APK, no rebuild: only the value on the Console changed between before and after



Before · groups_enabled = true



After · groups_enabled = false

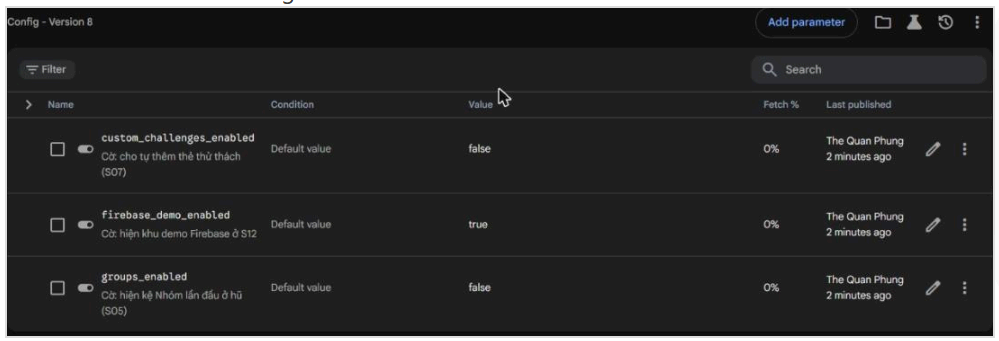


Before · custom_challenges_enabled = true



After · custom_challenges_enabled = false

Console · three feature flags



In-app defaults · used until a fetch succeeds

```
await config.setDefaults(<String, Object>{
  'quick_win_chips': jsonEncode(InAppRemoteConfig.defaultQuickWinChips),
  'groups_enabled': true,
  'custom_challenges_enabled': true,
  'discord_enabled': true,
  'discord_invite_url': InAppRemoteConfig.defaultDiscordInviteUrl,
  'discord_channel_map': '{}',
  'discord_greetings': jsonEncode(
    InAppRemoteConfig.defaultDiscordGreetings,
  ),
  'firebase_demo_enabled': true,
});
```

Fetch and cache · debug: every launch · release: 1 h cache

```
await config.setConfigSettings(
  RemoteConfigSettings(
    fetchTimeout: const Duration(seconds: 10),
    // Debug lấy ngay để demo "đổi trên Console, không build lại".
    minimumFetchInterval: kReleaseMode
        ? const Duration(hours: 1)
        : Duration.zero,
  ),
);
await config.fetchAndActivate();
} on Object catch (error) {
  debugPrint('[remote-config] dùng giá trị mặc định: $error');
}
```


docs/analytics-plan.md §10 · firebase/remote-config-defaults.json · evidence/remote-config/

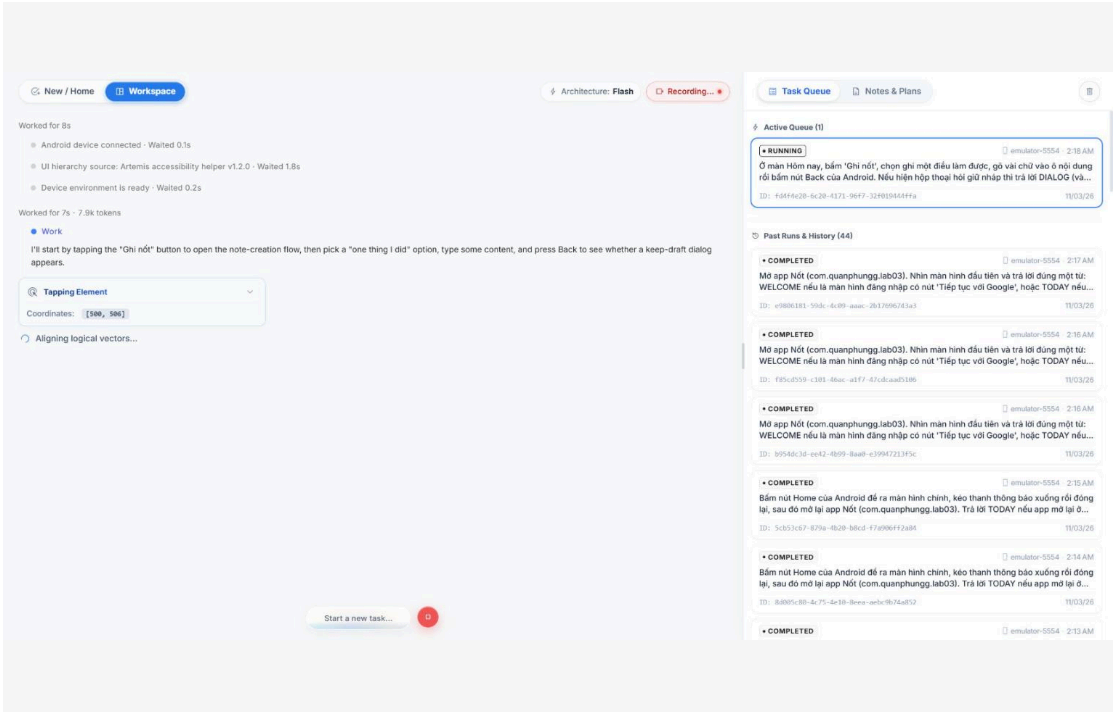
FPT Education
FPT UNIVERSITY

ARTEMIS (AI E2E testing)

ARTEMIS is Google's open-source AI agent that controls an Android device over ADB: it gets a goal in plain language, taps through the app and reports a value.

- Local server + LLM (`deepseek-v4.1-flash` via OpenRouter)
- Our pytest suite sends goals and asserts on the answer
- Locked to the app package, so it stays in Nốt

In Nốt: "save a note" must end with the agent reporting SAVED.



ARTEMIS Workspace: agent steps and run history

FPT Education
FPT UNIVERSITY

ARTEMIS (AI E2E testing)

WHY WE USE IT

The real app end to end
Real build, real Firebase, Android 15 emulator.

Also outside the app
Home button, notification shade, airplane mode.

Three runs per case
14 cases × 3 runs: 41 of 42 pass.

WHY NOT SOMETHING ELSE?

✕ Flutter `integration_test`
Stays inside our app: cannot press Home, open the shade or switch on airplane mode.

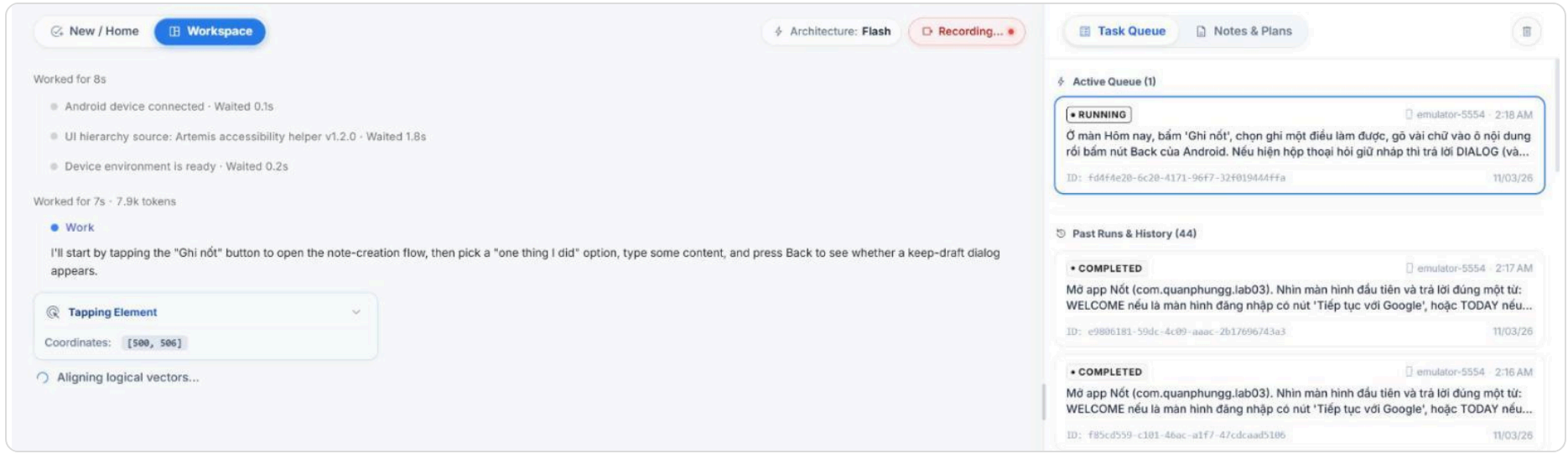
✕ Appium / Espresso scripts
Every tap is scripted by element ID, so a layout change breaks the test.

✕ Manual testing
Not repeatable 3 times per case with a recorded result.

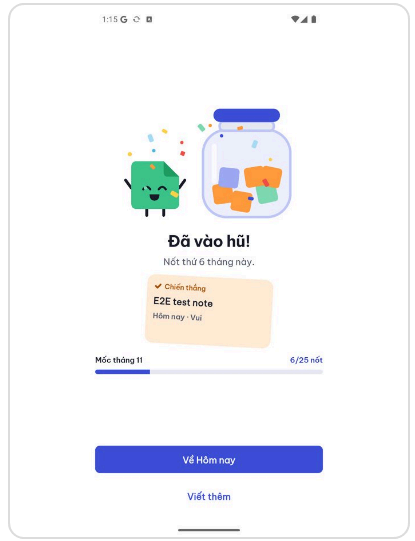
FPT Education
FPT UNIVERSITY

ARTEMIS (AI E2E testing)

Put the OpenRouter key in tools/artemis/.env, start the ARTEMIS server next to the emulator, then run `uv run pytest --runs 3` in e2e_test/.



1 Server running: agent steps live, run history



2 T02: the agent saved a note and reported SAVED

View demo and evidence →

ARTEMIS (AI E2E testing)

e2e_test/conftest.py :138–147

Send a goal, lock the app, read the answer

```
137
138     async def run(goal: str, expected: str, *, locked: bool = True, timeout: int
139         result = await client.run(
140             goal,
141             locked_app_package=app_id if locked else None,
142             expected_output=expected,
143             timeout=timeout,
144         )
145         assert result.succeeded, f"[run{attempt}] {result.error or result.status}
146         answer = (await reported(result)).strip().upper()
147         return answer, result
148
```

e2e_test/test_main_flows.py :39–48

T02: save a note → SAVED

```
38
39  async def test_t02_f1_save_a_note(ask) -> None:
40      """F1 nhánh chính: Today → Ghi nốt → viết → lưu → màn xác nhận."""
41      answer, result = await ask(
42          "Ở màn Hôm nay, bấm 'Ghi nốt'. Chọn ghi một điều làm được (không phải nhật
43          f"chọn một tâm trạng bất kỳ, gõ đúng chữ '{NOTE_TEXT}' vào ô nội dung, bỏ
44          "phần kể thêm ('Bỏ qua phần kể thêm') rồi bấm lưu. Khi màn hình xác nhận đ
45          "hiện ra, trả lời SAVED nếu thấy, ngược lại FAILED." + RULES,
46          "SAVED hoặc FAILED",
47      )
48      assert "SAVED" in answer, f"{answer!r} (trace {result.trace_id})"
49
```

e2e_test/test_main_flows.py :76–86

T06: airplane mode over adb

```
75
76  async def test_t06_f1_offline_note_is_kept_on_device(ask) -> None:
77      """F1 nhánh lỗi mạng: mất mạng thì lưu trên máy và báo cho người dùng biết.""
78      subprocess.run([ADB, "shell", "cmd", "connectivity", "airplane-mode", "enable
79      try:
80          answer, result = await ask(
81              "Máy đang mất mạng. Ở màn Hôm nay, bấm 'Ghi nốt', chọn ghi một điều l
82              f"tâm trạng bất kỳ, gõ '{NOTE_TEXT}' rồi lưu. Trả lời OFFLINE_BANNER
83              "báo nốt được lưu trên máy / đang chờ mạng (banner hoặc thông báo), S
84              "lưu như bình thường không nhắc gì đến mạng, FAILED nếu không lưu đượ
85              "OFFLINE_BANNER, SAVED hoặc FAILED",
86          )
87      finally:
```

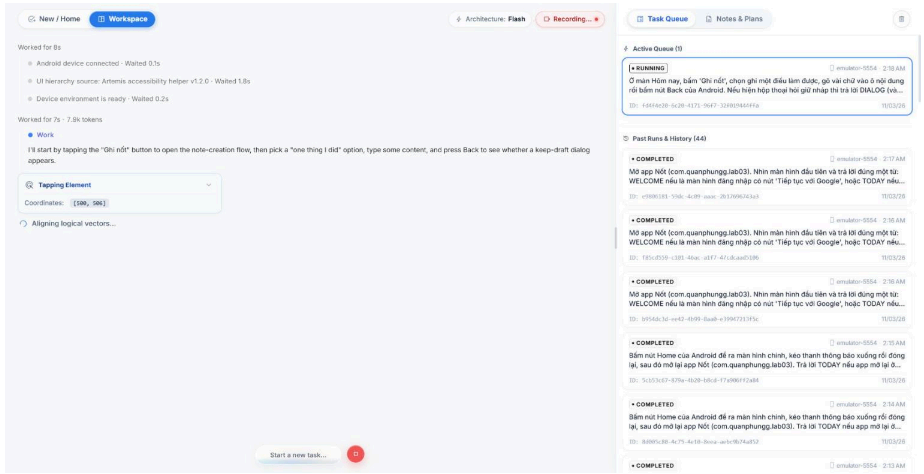
tools/artemis/config/artemis.jsonc :9–12

Model: deepseek-v4.1-flash

```
8       // ------------------------------------------------------------------------
9       "default": {
10        "provider": "openrouter",
11        "model": "deepseek/deepseek-v4.1-flash-1017",
12        "thinking_level": "medium",
13        "fallback": {
```

Real code on GitHub · FU-PRM-Projects/Lab-03 · branch main · key lines boxed in red

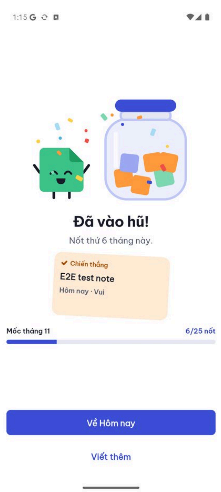
ARTEMIS E2E



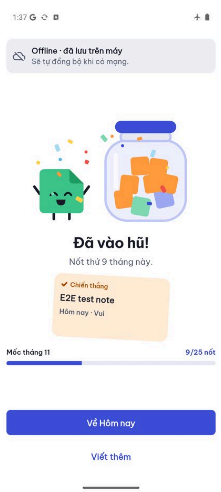
ARTEMIS Workspace · agent steps and run history (44 runs)

An AI agent (`deepseek-v4.1-flash` via OpenRouter) drives the app on the Android 15 emulator over ADB. Each case runs 3 times and asserts on the value the agent reports.

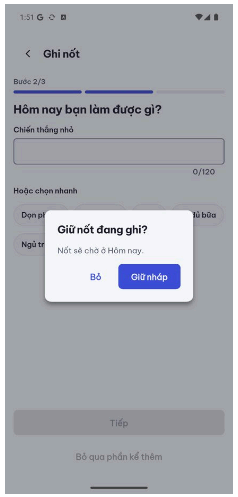
- **14 cases:** F1 save, empty, draft dialog, offline · F2 · F3 · tabs · flags · native Home and relaunch · analytics event · weekly card export
- **41 of 42 runs pass.** T06 (offline) timed out once.
- Three test bugs, not app bugs, were found on the way: two wrong assertions and a fixture that read the answer too early (fixed, re-run).
- Exploratory session (25 min, Pro): the agent reported 10 issues. Not run: sign-in and sign-out cases.



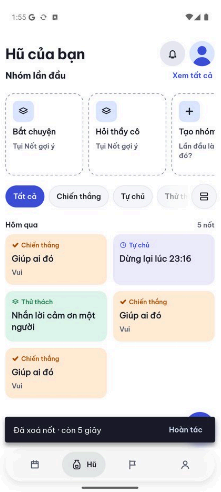
T02 · note saved



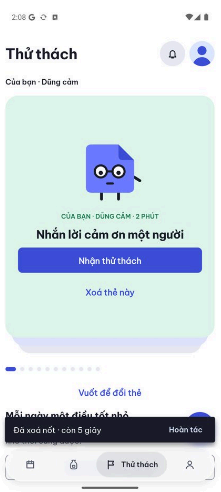
T06 · saved on device



T05 · keep-draft dialog



T03 · note deleted



T09 · challenge opened

docs/test-plan.md · e2e_test/ · evidence/artemis/

ARTEMIS: result per case

14 cases × 3 runs · 41 of 42 pass (98%) · T01–T12 full run 62 minutes

ID	Flow	Agent must report	Pass	Time / run
T01	Launch	`WELCOME or TODAY`	3/3	20–52 s
T02	F1 main path: save a note	`SAVED`	3/3	31–206 s
T03	Jar: open and delete the note	`DELETED`	3/3	256–391 s
T04	F1 error: empty note	`DISABLED`	3/3	55–56 s
T05	F1 error: Back with a draft	`DIALOG`	3/3 *	108–214 s
T06	F1 error: airplane mode (adb)	`OFFLINE_BANNER`	2/3	264–421 s
T07	F2 self-control	`SELF_CONTROL`	3/3	38–52 s
T08	F2 hard to stop: breathing	`BREATHING`	3/3	20–28 s
T09	F3 open a challenge, back	`DETAIL_THEN_BACK`	3/3	46–82 s
T10	Four tabs	`ALL_TABS`	3/3	33–40 s
T11	Settings: Remote Config row	`“Cờ tính năng”`	3/3	13–19 s
T12	Native: Home, shade, relaunch	`LOGGED_IN`	3/3 *	32–39 s
T13	Analytics: note_saved in logcat	`SAVED + event`	3/3 *	66–119 s
T14	Export the weekly card	`ERROR_MESSAGE`	3/3	41–71 s

T06 run 3 · real failure: timed out at 420 s, no answer from the agent. Root cause not found; no sign it is an app bug.

*** T05 run 3, T12:** our assertions were wrong (the app was right). Fixed, re-ran: 6 of 6.
*** T13 run 1:** the agent said SAVED, but our fixture read it before the last step was stored. Fixed (retry), re-ran: 3 of 3.

Error paths: T04, T05, T06
Outside the app: T06, T12
T14: no Storage bucket, so the expected result is the error message + Thử lại

docs/test-plan.md §5 · evidence/artemis/ junit-full-run · junit-rerun-t05-t12 · junit-t13-t14 · junit-rerun-t13

Semantics labels

41 files in lib/ set Semantics, including 17 of the 21 shared widgets

Mood choice · button, selected, one clear label `mood_option.dart`

```dart
return Semantics(
  button: true,
  selected: selected,
  label: 'Tâm trạng ${mood.label}',
  onTap: onTap,
  excludeSemantics: true,
  child: SizedBox.fromSize(
```

Today shortcuts · label and caption read together `shortcut_button.dart`

```dart
return Semantics(
  button: true,
  label: caption == null ? label : '$label, $caption',
  onTap: onTap,
  excludeSemantics: true,
```

Offline banner · live region, announced when it appears `offline_banner.dart`

```dart
return Semantics(
  liveRegion: true,
```

Why

- TalkBack reads one clear name per control, not the icon and text inside it.
- `selected` and `liveRegion` tell the reader about state and new messages.
- Test tools that read the accessibility tree can find a control by its name.

lib/widgets/ · mood_option.dart · shortcut_button.dart · offline_banner.dart

ARTEMIS: not run yet

Every other case has a result; these two need an email account made only for tests

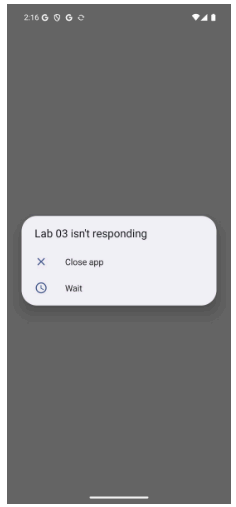
T15 · Sign-out, sign-in	Sign out, then sign in with an email test account (not Google: its picker shows a real name and email)	Needs a test account
T16 · Wrong password	Error message under the field, then sign in with the right password	Needs a test account

e2e_test/test_main_flows.py · docs/test-plan.md §4, §6

ARTEMIS: exploratory session

X01 · Pro profile · 25 min 38 s · 112 steps · the agent reported 10 issues; we have not checked them yet

#	Where	What the agent saw
X-01	Relaunch	Back from Today, reopen: “Lab 03 isn’t responding”, recovers after Wait
X-02	Jar, landscape	The + button covers “Xem tất cả”
X-03	Today, landscape	Tab bar covers the streak card; jar fills the screen
X-04	Jar vs Today	Notes this month: 22 on one, 21 on the other
X-05	Group list	Two group names cut with “…”
X-06	Journal editor	Empty or spaces only: Xong disabled, no reason shown
X-07	Jar	Tapping the jar picture does nothing
X-08	After saving	Blank “Đang tải” screen for a few seconds
X-09	Journey · Calendar	Mood legend does not say which colour is which
X-10	Journey · Year	12-month grid has no month names, cells tiny



X-01 · step 100



X-08 · step 13

Not checked yet. These are candidates from the agent. Some may have a reason (debug build, two screens loading at different times, one-line names in D-23).

Our side: the Pro profile answers without `report_task_status`, so the JUnit file shows a fixture error; the fixture now reads the final answer.

Clean-up: the runs left 7 test notes on the account; T03 and a manual pass removed them (back to 9 notes this month).

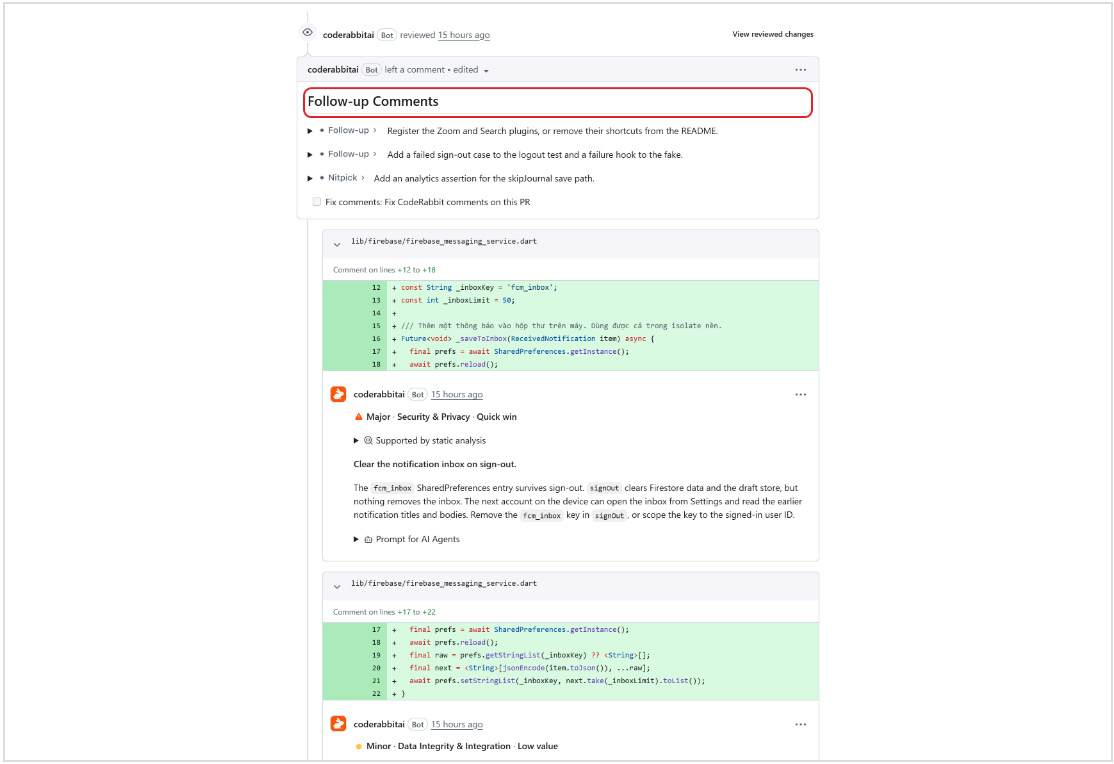
docs/test-plan.md §5 · evidence/artemis/x01-bugs.txt · evidence/artemis/x01/ · junit-x01.xml

CodeRabbit (AI code review)

CodeRabbit is an AI code review app for GitHub: it reads each pull request and leaves inline comments with a severity, a category and a suggested fix.

- Runs on every PR and on each new push
- Inline comments on the exact lines
- Severity (Major / Minor) and kind (security, correctness…)

In Nốt: 52 findings on PRs #28, #29 and #38.



CodeRabbit review on PR #38

FPT Education
FPT UNIVERSITY

CodeRabbit (AI code review)

WHY WE USE IT

A second reviewer every time

A small team cannot review every line twice.

It found real bugs

The FCM inbox survived sign-out (privacy).

We still judge each one

51 right, 1 wrong; each fix is its own commit.

WHY NOT SOMETHING ELSE?

✕ Human review only

Four students cannot read every line of every PR twice.

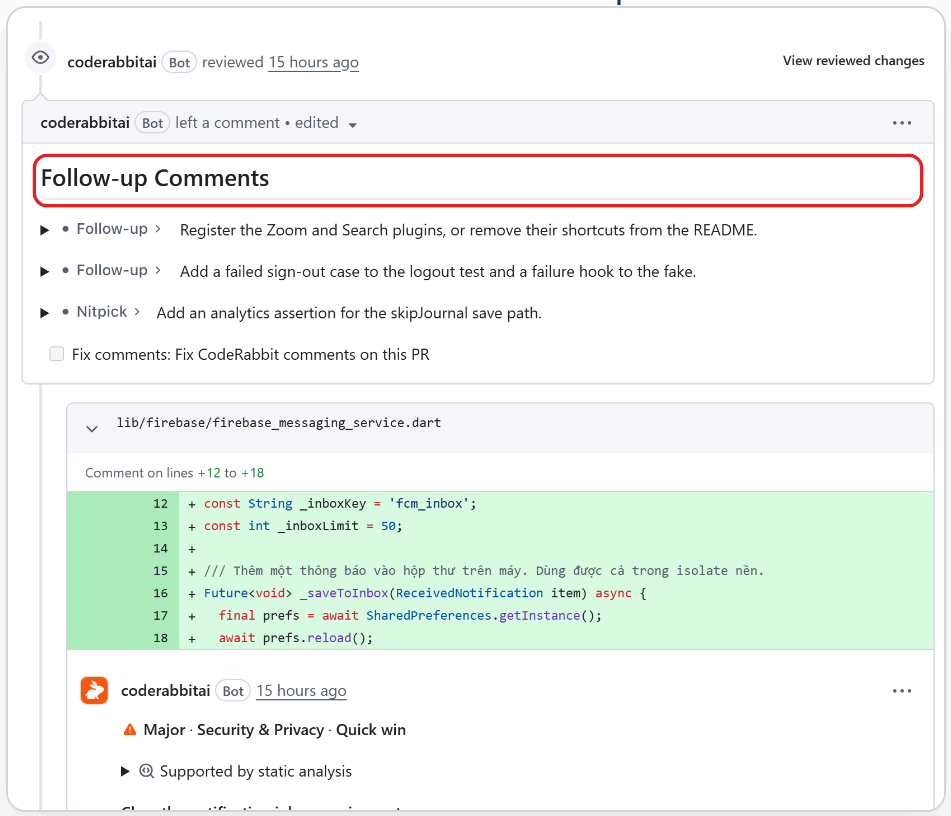
✕ `flutter analyze` only

Finds lint and type errors, not logic bugs like the inbox leak; CI still runs it.

FPT Education
FPT UNIVERSITY

CodeRabbit (AI code review)

Install the CodeRabbit app on the repository. It reviews each PR; we write a verdict for every finding in docs/code-review.md and fix the real ones in separate commits.



1 CodeRabbit reviews every PR 52 findings on #28, #29 and #38

Góp ý của AI	Kết luận	Ghi chú
Clear the notification inbox on sign-out.	Đúng, đã sửa	736453a: `signOut` xoá `fcm_inbox` và bump `inboxRevisionProvider`; có test ở `auth_view_model_test.dart` và `firebase_messaging_service_test.dart`.

2 We judge every finding 51 right, 1 wrong · verdicts in docs/code-review.md

```
fix: clear the FCM inbox on sign-out

The inbox lives in SharedPreferences and is shared by every account on the
device, so the next user could read the previous user's notifications in
Settings. Sign-out now clears it and bumps the inbox revision so an open
sheet reloads. Addresses CodeRabbit finding 38-bfeb007a.
```

```
fix: open the right screen when a foreground notification is tapped

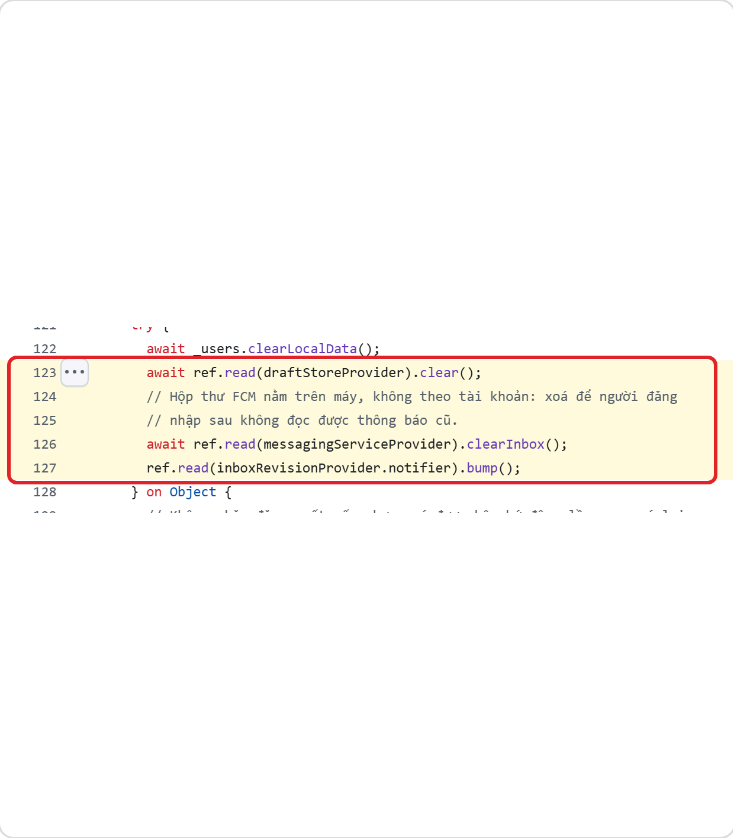
The heads-up shown while the app is open had no payload, and the tap
handler drops taps without one, so tapping it did nothing. Pass the route
from the message data as the payload, and let the FCM tap handler route it
too in case it is the handler the plugin keeps. Addresses CodeRabbit
finding 38-75b5e00c.
```

3 We fix with a commit 3 fixed so far; each message names its finding

View demo and evidence →

CodeRabbit (AI code review)

lib/viewmodels/auth_view_model.dart :123–127
Fixed **736453a**: clear the inbox on sign-out



```
122         await _users.clearLocalData();
123         await ref.read(draftStoreProvider).clear();
124         // Hộp thư FCM nằm trên máy, không theo tài khoản: xoá để người đăng
125         // nhập sau không đọc được thông báo cũ.
126         await ref.read(messagingServiceProvider).clearInbox();
127         ref.read(inboxRevisionProvider.notifier).bump();
128       } on Object {
```

lib/firebase/firebase_messaging_service.dart :101–107
Fixed **5e13651**: payload carries the route



```
100       if (n != null) {
101         await _local.show(
102           id: m.hashCode,
103           title: n.title,
104           body: n.body,
105           notificationDetails: const NotificationDetails(android: _channel),
106           payload: routeFromData(m.data),
107         );
108       }
```

lib/screens/challenges/create_challenge_sheet.dart :97–98
Rejected: **canSubmit** already checks **_saving**

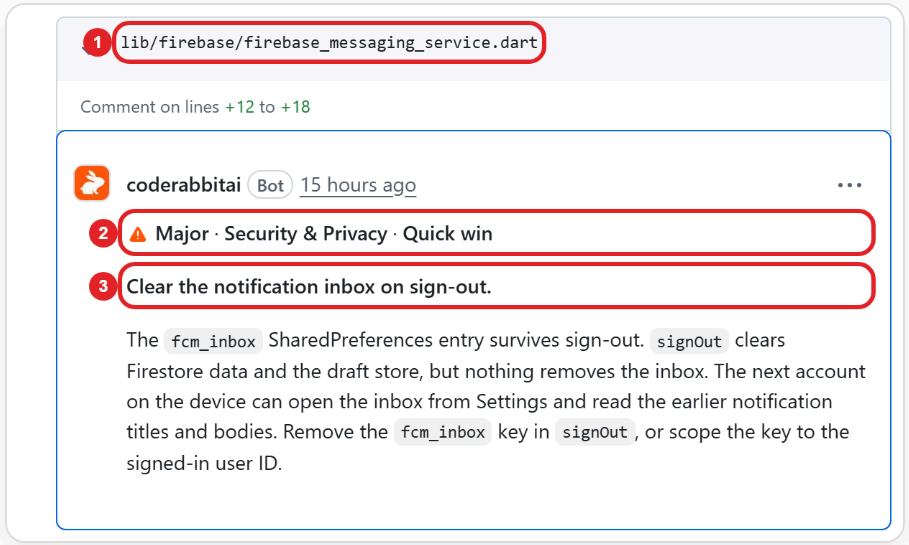


```
96        final theme = Theme.of(context);
97        final trimmed = _title.text.trim();
98        final canSubmit = !_saving && trimmed.isNotEmpty;
99
```

Real code on GitHub · FU-PRM-Projects/Lab-03 · branch main · key lines boxed in red

AI review: what we fixed

Real CodeRabbit comments on PR #38, marked in red, and the commit that fixed each one



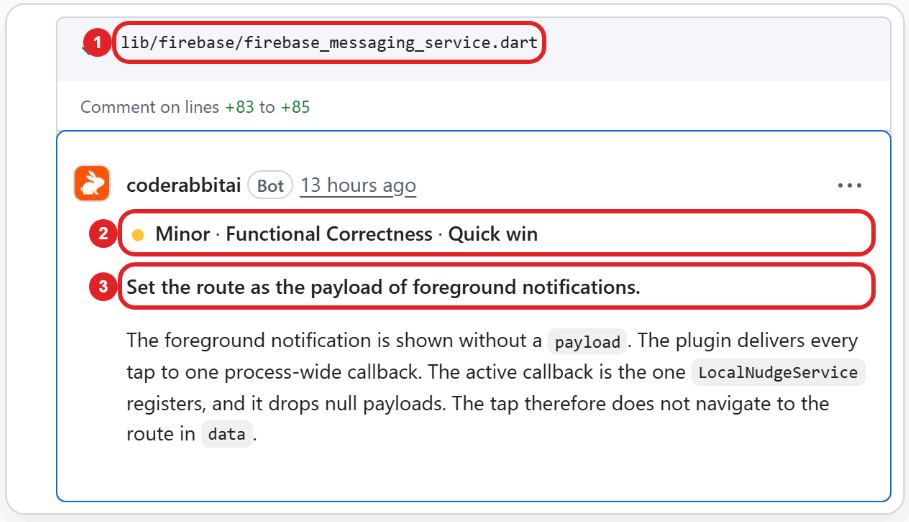
1 **WHERE**
firebase_messaging_service.dart, lines 12–18

2 **KIND**
Major · Security & Privacy

3 **FOUND**
The FCM inbox survives sign-out, so the next account can read old notifications

→

Fixed
`736453a`
Sign-out clears the inbox and the open sheet reloads. Two tests.



1 **WHERE**
firebase_messaging_service.dart, lines 83–85

2 **KIND**
Minor · Functional Correctness

3 **FOUND**
A notification shown while the app is open has no payload, so a tap opens nothing

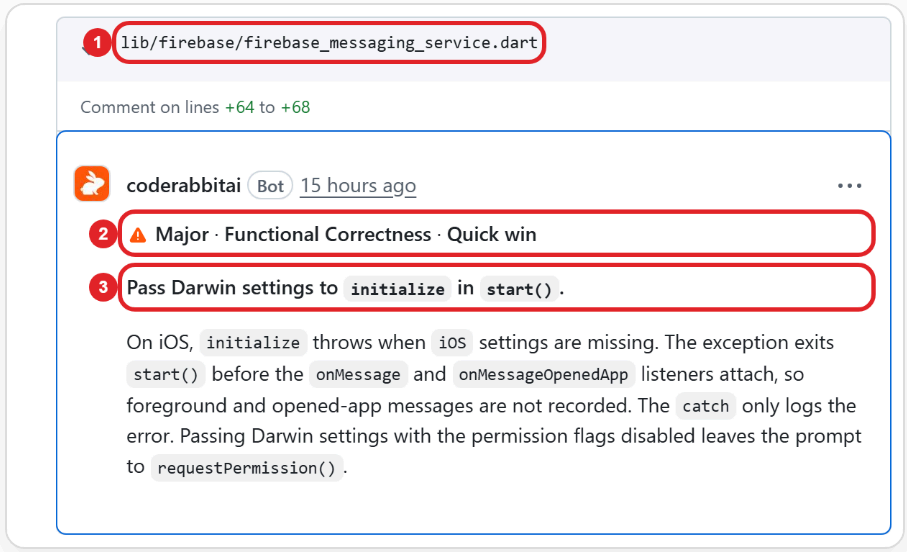
→

Fixed
`5e13651`
The payload carries the route; tapping opens that screen. Two tests.

docs/code-review.md · rows 38-bfeb007a, 38-75b5e00c

AI review: what we did not fix

Every comment was checked against the code before deciding



1 **WHERE**
firebase_messaging_service.dart, lines 64–68

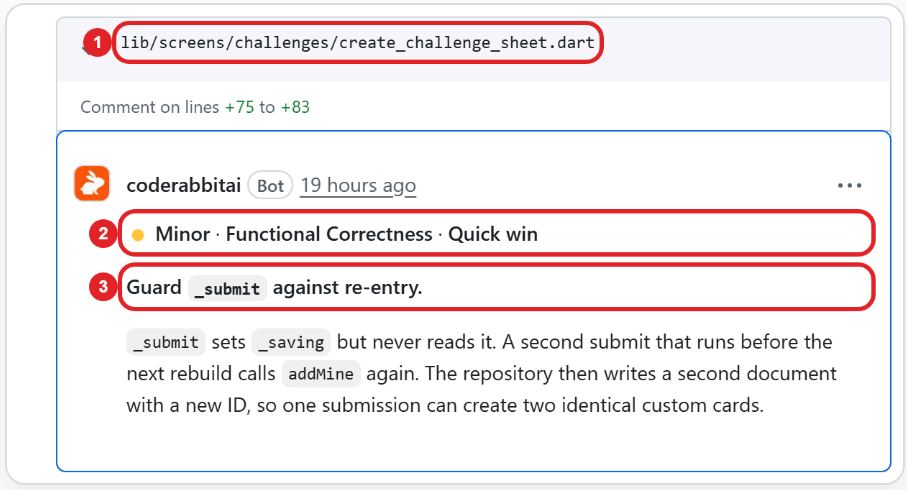
2 **KIND**
Major · Functional Correctness

3 **FOUND**
Notification setup has no iOS (Darwin) settings, so it would fail on iOS

→

Won't fix

Right for iOS, but the app ships on Android and web only. Major is too strong here.



1 **WHERE**
create_challenge_sheet.dart, lines 75–83

2 **KIND**
Minor · Functional Correctness

3 **FOUND**
_submit sets _saving but never reads it, so one tap could add two cards

→

Rejected

The AI is wrong: canSubmit = !_saving && … already disables the button while saving.

docs/code-review.md · rows 38-cfffb9a5, 29-c0bdd08c

Lessons learned

1

Test on the real project early

Emulator has no Analytics or FCM

2

Design docs can disagree

Pick one, log it · D-11

3

Rename what users misread

"Bỏ qua phần kể thêm" · D-28

4

Check the AI tester too

4 of 5 failures: our own asserts

5

Read AI review before merging

1 of 40 bugs fixed at merge

FPT Education

FPT UNIVERSITY



Thank you!

AI tools used **Claude Code** code, docs, slides **CodeRabbit** PR review **ARTEMIS** E2E agent **DeepSeek** ARTEMIS model